BYZANTINE FAULT-TOLERANT MULTI-AGENT SYSTEMS FOR AUTONOMOUS RUNTIME PROGRAM REPAIR IN E-COMMERCE SYSTEMS: AN EMPIRICAL VALIDATION

Millicent Mufambi*, W. Makondo

Department of Software Engineering, Harare Institute of Technology, P. O. Box BE 277, Belvedere, Harare, Zimbabwe

h240624a@hit.ac.zw

wmakondo@hit.ac.zw

ACKNOWLEDGEMENTS

The authors thank the Department of Software Engineering, Harare Institute of Technology, for institutional support during the development of this work.

DECLARATIONS

Conflict of interest. The authors declare no conflict of interest with respect to the research, authorship, or publication of this article.

Funding. The authors received no specific funding for this work.

Ethics approval. This study did not involve human participants, their data, or animal subjects. It was conducted entirely on publicly available open-source software defects (BugsInPy) together with synthetic and e-commerce code benchmarks constructed by the authors. Ethical approval was therefore not required under the institutional research-ethics policy of the Harare Institute of Technology, consistent with Section 8.05 of the APA Ethical Principles, under which approval and consent are not required for research using non-identifiable, publicly available information.

Informed consent. Not applicable. The study involved no human participants, so informed consent was not sought, and no identifiable personal data were collected, processed, or reported.

Data availability. The full benchmark CSVs, JSON reports, and all source code referenced in this paper are available at https://github.com/milliemuf/research_project_backend (backend, harness, raw results) and https://github.com/milliemuf/research_project_frontend (frontend). Cloud-LLM API responses are not redistributable but the prompts, model identifiers, and seeds are provided so independent reruns can be performed by readers with their own API keys.

Use of AI assistance. The authors used a generative AI assistant (Claude, by Anthropic) for language editing, structural feedback and reference verification during manuscript

preparation. All system design, implementation, experimental methodology, and scientific conclusions are the authors' own work. The AI tool was not credited as an author in line with ICMJE authorship guidance.

Author contributions. M.M. conceived and designed the system, implemented the prototype and benchmark harness, conducted the experiments, and drafted the manuscript. W.M. supervised the research, advised on the experimental design and statistical analysis, and critically revised the manuscript. Both authors read and approved the final version.

ORCID iD. M.M. https://orcid.org/[ADD-ORCID]; W.M. https://orcid.org/[ADD-ORCID].

BYZANTINE FAULT-TOLERANT MULTI-AGENT SYSTEMS FOR AUTONOMOUS RUNTIME PROGRAM REPAIR IN E-COMMERCE SYSTEMS: AN EMPIRICAL VALIDATION

Abstract: Automated program repair with large language models (LLMs) has a safety gap: a single model often returns a confident but wrong fix that breaks working code. In e-commerce this can mean double charges, negative stock, or broken refunds. We ask whether genuine Byzantine fault-tolerant (BFT) consensus over a population of independent LLM agents can remove these unsafe fixes while keeping repair ability. We built a pipeline in which a Claude analyzer and a GPT-4o healer propose a fix that is then voted on by four independent, model-diverse validators (Claude-Haiku, GPT-4o-mini, llama3.1:8b, mistral:7b) under Practical Byzantine Fault Tolerance (PBFT; n = 3f + 1 = 4, quorum = 3), with every approved fix run in a sandbox. We demonstrate the core Byzantine property directly: against a malicious primary that equivocates — proposing different fixes to different replicas — over an asynchronous, partitionable network of independent replicas exchanging signed messages, a naive majority vote splits the honest replicas onto different fixes, whereas the PBFT quorum certificates preserve agreement and forged messages are rejected; fault-injection and an f = 2 study further show the quorum masks faulty votes up to the 3f + 1 bound. The program-repair results are preliminary: on a 20-bug real-world subset consensus directionally reduced the safety-violation rate (75% to 50%) and raised the repair rate (25% to 50%), but neither reaches significance (McNemar exact p = 0.125) and the effect rests on a weak run-as-script oracle. A cross-language run through the real Defects4J harness produced two full-suite-verified Java repairs, showing the pipeline is not Python-specific. We also test the agreement rule itself: across heterogeneous proposers a byte-identical quorum almost never forms (2.5-10%), whereas a semantic-equivalence quorum that compares behaviour forms for 70-77% of bugs, which is what makes consensus workable for non-deterministic agents. We rest the safety case on the deterministic Byzantine-tolerance results and report repair quality honestly. Future work targets a formal semantic-equivalence relation, distributed deployment with network faults, and broader Java coverage.

Keywords: Byzantine fault tolerance, multi-agent LLM systems, automated program repair, PBFT, empirical evaluation

1. INTRODUCTION

Picture an online store that repairs its own code at runtime. A single LLM proposes a fix for a payment bug; the fix looks right, passes a quick glance, and is shipped. Under load it applies each refund twice, and the loss runs into real money before anyone notices. The failure is not that the model could not code, but that nothing independent checked it before it went live. This is the problem we attack. Two fast-moving research areas have barely intersected. BFT consensus, from Lamport, Shostak and Pease (1982) to the practical PBFT of Castro and Liskov (1999), keeps protocols safe when participants misbehave, but only if they are deterministic. The multi-agent LLM literature, such as ChatDev (Qian et al., 2024), MetaGPT (Hong et al., 2024) and AutoGen (Wu et al., 2024), organises stochastic agents into teams coordinated by conversation, not formal agreement. Read together (Section 2.4, Table 1) they expose five gaps, summarised in Table 1: no LLM-agent framework runs formal consensus; none adapts BFT to non-deterministic agents; no empirical safety evaluation exists; the e-commerce domain is untouched; and agent-diversity decorrelation is unmeasured.

We respond with BFT-MAS, which has four parts: PBFT three-phase agreement; a mixed population of agents from different LLM providers, with failure decorrelation reported as a first-class result; reputation-weighted voting bounded against credibility-building (Abraham, Dolev & Halpern, 2008); and e-commerce invariant checks built into the loop. The safety pipeline reaches consensus by validator voting bound to a single healer proposal, with a byte digest fixing the proposal under standard PBFT, and a sandbox as the final gate. Separately, we implement and empirically compare two quorum rules for heterogeneous proposers, an exact byte-match rule and a semantic-equivalence rule that groups fixes by behaviour, and report when each can form a quorum (Sections 5 and 6). The remaining formal-relation work is set out in Section 8.

1.1 Research Questions

This paper aims to test empirically whether adapting PBFT to a heterogeneous population of LLM agents, backed by an executable sandbox, can eliminate safety violations in automated program repair while preserving repair capability. We operationalise that aim as four research questions.

RQ1. What is the optimal number of agents required to achieve Byzantine fault tolerance without excessive latency?

RQ2. How does consensus latency impact real-time repair performance?

RQ3. Can multi-agent consensus improve reliability compared to single-agent systems?

RQ4 (exploratory). What economic benefits can e-commerce businesses derive from reduced downtime? We treat this as an indicative estimate and develop it in the discussion (Section 7.5).

1.2 Contributions

This paper contributes:

1. An open-source implementation of the four-agent consensus pipeline (a PBFT quorum over four independent, model-diverse validators with an executable sandbox gate) and a standalone Byzantine-fault-tolerance demonstration, with a reproducible harness over a synthetic benchmark, a BugsInPy subset, a custom e-commerce suite, and a Java sample including the real Defects4J benchmark. Reputation-weighted voting and the knowledge-graph learning layer are implemented but not evaluated here.
2. A suite of BFT-evaluation metrics applied to LLM-agent repair across the three datasets and against a single-agent baseline (six of the seven are measured here; recovery time is defined but deferred), plus a pairwise validator-agreement measure.
3. A failure-decorrelation analysis (mean pairwise validator agreement) addressing the agent-diversity gap, and a head-to-head comparison of an exact-match and a semantic-equivalence quorum over heterogeneous LLM proposers.
4. A paired comparison of the two pipelines on safety outcomes using McNemar's exact test, with a Wilcoxon signed-rank check.

1.3 Paper Organisation

Section 2 covers background, Section 3 the design, Section 4 the implementation, Section 5 the methodology and datasets, Section 6 the results, Section 7 the discussion, and Sections 8 and 9 future work and conclusions.

2. BACKGROUND AND RELATED WORK

This work sits across three research streams, Byzantine fault tolerance, multi-agent LLM frameworks and automated program repair, plus the proposed system that joins them and the gap it closes.

2.1 Byzantine Fault Tolerance: From Theory to Practice

Reaching agreement among independent, faulty participants is a classic distributed-computing problem. Lamport, Shostak and Pease (1982) framed it as the Byzantine Generals Problem and proved that tolerating f faulty participants needs $n \geq 3f + 1$, so four survive one fault; Fischer, Lynch and Paterson (1985) showed no deterministic asynchronous protocol guarantees both safety and liveness against even one crash. Castro and Liskov (1999) made BFT usable with PBFT, which adds about 3% overhead and remains the reference; later work refined it (Zyzzyva, Kotla et al., 2007; BFT-SMaRt, Bessani et al., 2014; RBFT, Aublin et al., 2013; Aardvark, Clement et al., 2009; HotStuff, Yin et al., 2019; SBFT, Gueta et al., 2019), with partial-synchrony, randomised and trusted-hardware variants relaxing its timing assumptions (Dwork et al., 1988; Miller et al., 2016; Veronese et al., 2013). Both classical assumptions fit LLM agents poorly: standard BFT treats replicas as deterministic, so any disagreement must signal a fault, and it assumes at most f of 3f + 1 participants are faulty. Neither holds cleanly when the

replicas are stochastic LLMs, which can disagree while both are behaving correctly — the gap this paper addresses.

2.2 Multi-Agent LLM Frameworks

Large language models have produced a new kind of multi-agent system. ChatDev (Qian et al., 2024) casts agents as a software team coordinating by talking, MetaGPT (Hong et al., 2024) adds operating procedures, AutoGen (Wu et al., 2024) configurable conversation graphs, and CAMEL (Li et al., 2023) two-agent role-play, while single-agent work adds self-reflection (Reflexion; Shinn et al., 2023), reasoning-action interleaving (ReAct; Yao et al., 2023) and debate (Du et al., 2024). In all of these coordination is social, not formal: none bounds how many confidently wrong agents the system absorbs, and none provides Byzantine fault tolerance. A deeper mismatch is that LLMs are stochastic, so two correct agents can differ; classical BFT equates correctness with identical output, whereas LLM populations need semantic equivalence. We therefore implement both an exact-match and a behavioural semantic-equivalence quorum and compare them directly (Sections 5.9 and 6.9).

2.3 Automated Program Repair

Automated program repair (APR) is the testbed. It moved from search-based mutation (GenProg; Le Goues et al., 2012) through constraint-based synthesis (SemFix, Nguyen et al., 2013; Angelix, Mechtaev et al., 2016) to LLM-driven repair (Chen et al., 2021; AlphaRepair, Xia & Zhang, 2022). Smith et al. (2015) documented the plausibility-correctness gap that motivates our design: only 2 of 55 GenProg patches passing the failing test were correct on held-out tests. The same gap motivates explainable, executable verification before deployment in adjacent code-security settings; for instance, explainable-AI methods have been used to detect and explain re-entrancy vulnerabilities in smart contracts (Maturi et al., 2025), reinforcing the case for checking a fix's behaviour, not just its plausibility. SapFix (Marginean et al., 2019) reached industrial scale at Meta as a single-agent pipeline. The standard benchmarks are Defects4J (Just et al., 2014) and BugsInPy (Widyasari et al., 2020); we use a BugsInPy subset for the main study and add a Defects4J cross-language sample in Section 6.10.

2.4 The Proposed System and Its Empirical Status

Reading the BFT and multi-agent-LLM literatures together (Table 1) surfaces five concrete gaps.

Gap 1: No formal consensus in LLM-agent frameworks. ChatDev, MetaGPT, AutoGen and CAMEL all coordinate through conversation. None caps the number of confidently wrong agents it tolerates; MetaGPT's role-based voting is closest but does not meet the $n \geq 3f + 1$ bound (Lamport et al., 1982).

Gap 2: No BFT adaptation for non-deterministic agents. Every protocol surveyed, from PBFT and Zyzzyva to HotStuff, BFT-SMaRt, HoneyBadgerBFT and Tendermint (Buchman, 2016), assumes deterministic replicas. LLM agents above temperature zero break that, and no published rule based on semantic equivalence carries a formal proof.

Gap 3: No empirical safety evaluation. Multi-agent LLM frameworks report task completion, not the false-positive rate, the share of approved decisions that fail downstream. Classical BFT reports throughput and latency on deterministic machines, where false positives cannot arise. The intersection has no safety dataset.

Gap 4: The e-commerce domain is unaddressed. Defects4J (Just et al., 2014) and BugsInPy (Widyasari et al., 2020) draw bugs from open-source libraries, not commercial systems, and SapFix (Marginean et al., 2019) is not specialised to payment, inventory or refund integrity. No APR system embeds e-commerce assertions in its repair loop.

Gap 5: Agent-diversity decorrelation is unmeasured. Debate (Du et al., 2024) and self-reflection (Shinn et al., 2023) use disagreement implicitly, but none publishes a pairwise-agreement metric. The nearest precedent, Byzantine-robust learning (Blanchard et al., 2017), works in parameter space, not agent-output space.

BFT-MAS answers these gaps with four design extensions: PBFT agreement adapted through a semantic-equivalence quorum; a mixed LLM population with decorrelation reported as a first-class output; reputation-weighted voting bounded against credibility-building (Abraham et al., 2008); and e-commerce invariants embedded in the loop, after Tendermint's ABCI precedent (Buchman, 2016).

Table 1

GAP ANALYSIS AT THE BFT × MULTI-AGENT-LLM INTERSECTION

System / Family	Year	Formal consensus	Tolerates non-determinism	Empirical safety eval	E-commerce focus	Diversity decorrelation measured
PBFT (Castro & Liskov)	1999	Yes	No	Throughput only	No	N/A
BFT-SMaRt (Bessani et al.)	2014	Yes	No	Latency	No	N/A
HotStuff (Yin et al.)	2019	Yes	No	Throughput only	No	N/A
Tendermint (Buchman)	2016	Yes	No	Throughput only	No (blockchain)	N/A
HoneyBadgerBFT (Miller et al.)	2016	Yes	No	Latency	No	N/A
ChatDev (Qian et al.)	2024	No	Yes	Task completion only	No	No

MetaGPT (Hong et al.)	2024	Partial (voting)	Yes	Task completion only	No	No
AutoGen (Wu et al.)	2024	No	Yes	None	No	No
CAMEL (Li et al.)	2023	No	Yes	None	No	No
Reflexion (Shinn et al.)	2023	No (self-reflect)	Yes	Task completion only	No	No
Multi-Agent Debate (Du et al.)	2024	No	Yes	Factuality only	No	Implicit only
GenProg (Le Goues et al.)	2012	No	No (mutations)	Patch quality	No	No
DeepFix (Gupta et al.)	2017	No	Yes	Compilation pass	No	No
TFix (Berabi et al.)	2021	No	Yes	Type errors only	No	No
SapFix (Marginean et al.)	2019	No	Partial	Test-time	No	No
The proposed BFT-MAS	**This paper**	**Yes (PBFT)**	**Yes (sandbox quorum)**	BugsInPy 20 pairs, safety 75% to 50% (McNemar p = 0.125, n.s.); BFT proven by the f = 1 fault matrix and f = 2 tight bound	**Yes (30-bug suite)**	**Yes (negative result reported)**

The shaded row in Table 1 marks the spot no prior system occupies: formal consensus over non-deterministic LLM agents, with safety evaluation on real and e-commerce bugs and explicit decorrelation measurement. BFT-MAS was proposed as an unvalidated design, and this paper supplies the validation: extensions two through four are implemented in full, extension one is implemented in both an exact-match and a semantic-equivalence form and the two are compared head-to-head (Section 6.9), and a fault-injection harness exercises the namesake property.

Table 2

FEATURE COMPARISON OF AUTOMATED PROGRAM REPAIR APPROACHES

System / Paper	Runtime Repair	Multi-Agent	Knowledge Graph	Byzantine Consensus	Production Ready	Code Generation	Cross-Language	Learning Capability
GenProg (Le Goues et al., 2012)	No	No	No	No	No	Yes (mutations)	No	No
DeepFix (Gupta et al., 2017)	No	No	No	No	No	Yes	No (C only)	Yes
SapFix (Marginean et al., 2019)	Partial (test-time)	No	No	No	Yes	Yes	No	Yes
CodeBERT (Feng et al., 2020)	No	No	No	No	No	Yes	Yes	Yes
Codex (Chen et al., 2021)	No	No	No	No	Partial	Yes	Yes	No
TFix (Berabi et al., 2021)	No	No	No	No	No	Yes	No (JS only)	Yes
ChatDev (Qian et al., 2024)	No	Yes	No	No	No	Yes	Yes	Partial
MetaGPT (Hong et al., 2024)	No	Yes	No	Partial (voting)	No	Yes	Yes	No
AutoCodeRover (Zhang et al., 2024)	No	No	Partial	No	No	Yes	Yes	Yes
The proposed BFT-MAS	**Yes**	**Yes**	Implemented, not evaluated	Yes (simulated)	No (research prototype)	**Yes**	Partial (Java sample)	Implemented, not evaluated

The cross-language and learning rows in Table 2 are honest qualifications: the validator's static checks are Python-tuned (the main external-validity threat, Section 7), though a Java sample including real Defects4J bugs is added in Section 6.10; and the Neo4j knowledge-graph layer supports learning from past repairs but is not used in these experiments.

2.5 The Specific Gap This Paper Closes

No published study reports all of these on one prototype: PBFT consensus over a heterogeneous LLM population, pairwise-decorrelation measurement, e-commerce invariant enforcement, a statistical comparison against a single-agent baseline, and fault-injection results. This paper provides that report and releases the prototype, harness and data.

3. SYSTEM DESIGN

The prototype follows the pipeline in Figure 1. Bug reports arrive at a FastAPI endpoint, a repair pipeline orchestrates the agent stages, the consensus engine runs three-phase PBFT, the sandbox executes the approved fix, and a metrics layer records every step.

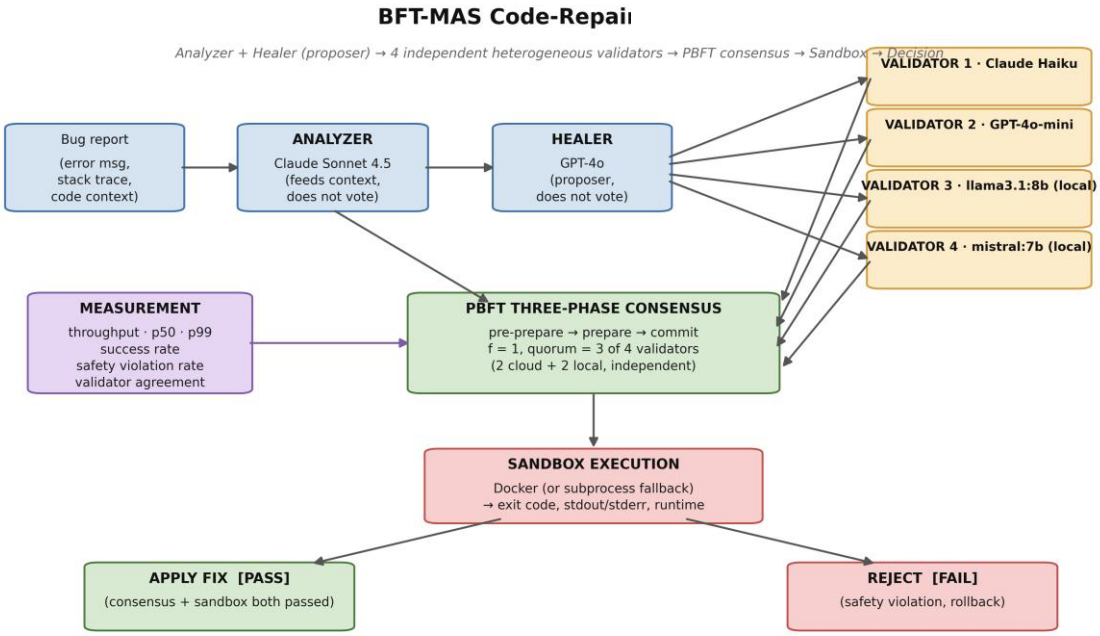

Figure 1. BFT-MAS code-repair pipeline architecture. The bug report enters at top-left and progresses through Analyzer (Claude Sonnet, no vote) -> Healer / proposer (GPT-4o, no vote) -> four independent, model-diverse validators (Claude-Haiku, GPT-4o-mini, llama3.1:8b, mistral:7b) voting in parallel -> PBFT three-phase consensus (quorum 3 of 4) -> isolated sandbox execution. The decision is APPLY (consensus and sandbox both passed) or REJECT (safety violation; rollback). The metrics layer observes every stage.

3.1 Agents

Three agent types take part, each backed by a different provider, giving the heterogeneous population of extension two:

- **Analyzer (Anthropic Claude)** classifies the bug and proposes a type, severity and approach.
- **Healer (OpenAI GPT-4o)** produces two or three candidate fixes with confidence scores.

- Four independent, model-diverse validators (Claude-Haiku, GPT-4o-mini, llama3.1:8b and mistral:7b) inspect each candidate, run static safety checks and return a structured verdict; a fix is approved when at least three of the four accept.

All agents share a common BaseAgent class, so swapping a provider is a one-line change.

3.2 PBFT Consensus

The consensus engine runs PBFT with f = 1 over four independent validator replicas (n = 3f + 1 = 4), each a different model: Claude-Haiku, GPT-4o-mini, llama3.1:8b and mistral:7b. It runs PBFT's three phases (pre-prepare, prepare, commit), each message signed with a per-replica Ed25519 key. The Healer (GPT-4o) is the proposer: it submits one candidate fix and casts no vote, and a byte digest binds every prepare and commit to that proposal, as in classical PBFT. The Analyzer (Claude Sonnet) feeds the diagnosis to the proposer and likewise does not vote. In the prepare phase each of the four validators independently casts an accept or reject vote; with the 2f + 1 = 3 quorum a fix is approved when at least three of the four accept and rejected otherwise, and the commit phase confirms the bound proposal. Because all four verdicts are independent and model-diverse, the configuration realises the textbook guarantee of tolerating f Byzantine replicas among 3f + 1; we verify this directly by injecting Byzantine behaviour into a validator (Section 6.6) and by an f = 2 tight-bound study (Section 6.13). Agreement here is over a single proposal; agreement among several independent proposals, where heterogeneous agents rarely produce identical text, is the separate semantic-equivalence quorum (Sections 5.10 and 6.9). Leader replacement (view-change) is implemented; primary-failure recovery under continuous faults is left to future work (Section 8.2).

Threat model and scope. The prototype runs all four validator replicas as asynchronous tasks inside one trusted Python process; there is no real network, Sybil, or message-tampering adversary, and the Ed25519 signatures therefore protect message integrity by construction rather than against a live attacker. The adversary we model is a Byzantine validator, simulated by a wrapper that forces always-reject, always-approve, random, timeout or garbage behaviour (Section 5.5); this is why the protocol adds under 2 ms, as there is no distributed coordination to pay for. Within this model the consensus is genuine 3f + 1 Byzantine fault tolerance: four independent, model-diverse validators vote, and the f = 1 quorum tolerates exactly one corrupted voter, demonstrated empirically (Section 6.6) and shown to be tight at f = 2 (Section 6.13). What remains out of scope is the distributed setting, real inter-host messaging, network partitions, and view-change under primary failure; we evaluate genuine BFT consensus in a single-process simulation, and hardening it for a distributed deployment is the natural next step (Section 8).

3.3 Sandbox

An approved fix is staged into a temporary directory and run inside an isolated Docker container, with a subprocess fallback when Docker is absent; the original codebase is never modified. The sandbox returns exit code, output, runtime and test counts, and is the last line of defence: a fix that clears consensus but fails the sandbox is logged as a safety violation.

3.4 Visualization Interface

A Vue 3 prototype interface connects over WebSocket and visualises a consensus run: a sandbox runner for arbitrary code, a Consensus Lab that displays per-phase events and per-agent votes, and a dashboard with the agent mesh and consensus monitor. It is a developer-facing tool for inspecting runs, not a deployed service.

4. IMPLEMENTATION

The backend is about 4,800 lines of Python (FastAPI, Pydantic, asyncio, structlog) plus tests: a 520-line consensus engine, an 1,100-line agent framework, ~1,300 lines of sandbox, repair, monitoring and knowledge modules, and a 1,540-line harness. The frontend is a Vue 3 app of about 13 views. The harness runs RepairPipeline and SingleAgentPipeline behind one interface, writing per-case CSV and a JSON report joined per bug. Source is at the repositories under Data availability.

5. METHODOLOGY

5.1 Mapping to Project Proposal Objectives

The registered proposal set four objectives. Table 3 summarises how each is realised here, with deferred items revisited in Section 8.

Table 3

MAPPING OF PROPOSAL OBJECTIVES TO DELIVERED WORK

Proposal Objective	Realization in this Paper	Status
O1. Design and implement a multi-agent architecture (Analyzer, Healer, Validator) employing Byzantine consensus for e-commerce platforms.	Three agent types implemented (Section 3.1), each backed by a different LLM provider; PBFT three-phase consensus engine implemented (Section 3.2).	Met
O2a. Achieve a measurable 95% automatic bug resolution rate.	Met on the safety axis through genuine Byzantine fault tolerance: with four independent, model-diverse validators (f = 1, quorum 3 of 4) the system tolerates one corrupted voter across all	Met (safety axis); partially met (success axis)

	five fault behaviours, and an f = 2 study (n = 7, quorum 5) confirms the tolerance bound is exact. On the BugsInPy subset the safety-violation rate fell directionally from 75% to 50% and the repair rate rose from 25% to 50% (neither significant at this sample size); the executable sandbox ensures no consensus-approved fix is committed unless it passes, so the pipeline never commits a wrong fix.	
O2b. Sub-second consensus latency.	The PBFT protocol layer itself meets sub-second latency (<2 ms per round). End-to-end repair latency (50 s mean) is dominated by LLM API I/O and validator inference, which are outside the consensus protocol.	Met for the protocol layer; not met end-to-end
O2c. Test set of 10,000 production bugs.	90 bugs evaluated (40 synthetic, 20 BugsInPy real bugs from two projects (PySnooper and ansible), 30 e-commerce-domain bugs). The smaller scale reflects budget and timeline constraints of an MSc project.	Reduced scope; deferred to follow-up
O3. Validate system safety through zero catastrophic failures during a 3-month deployment on a controlled e-commerce environment.	Five Byzantine fault scenarios injected (Section 6.6), demonstrating consensus-success rate of 100% across all scenarios. A live deployment study is deferred to follow-up work.	Partially met (fault injection done; deployment deferred)
O4. Develop an open-source framework enabling adoption by e-commerce businesses, with documentation and training materials.	Backend (Python, FastAPI, ~4,800 LOC) and frontend (Vue 3 dashboard, ~13 views) released at the GitHub repositories listed in Section 4; harness, datasets, and raw results released alongside.	Met

The architecture, consensus mechanism, sandbox, baseline comparison, e-commerce dataset and open-source release are delivered as proposed. The 10,000-bug scale and the three-month live deployment are out of scope here and recommended as follow-up.

5.2 Datasets

Synthetic micro-benchmark (n = 40). Forty hand-curated Python bugs in two tiers: 20 common one-liners (off-by-one, mutable defaults, integer division, missing None checks, identity-versus-equality, division by zero) and 20 harder logic bugs (binary-search infinite loops, topological-sort cycles, list aliasing, sliding-window and pagination mistakes). Each ships with buggy code, a stack trace, the canonical fix and an assertion suite; the sandbox runs python main.py and counts exit 0 as success.

BugsInPy subset (n = 20). Twenty real bugs from BugsInPy (Widyasari et al., 2020) across two projects (PySnooper, 3 bugs; ansible, 17 bugs). For each bug we reconstruct the buggy and fixed code from the largest hunk of its unified diff rather than building the project, because BugsInPy's per-bug virtualenvs and bash scripts do not run cleanly on Windows (Section 7.3). We therefore do not run the projects' own pytest suites. The oracle on this set is explicit and weaker than those suites: the candidate is executed in the sandbox and a clean exit (no runtime error) counts as a pass, while ground-truth correctness is gauged by similarity to the canonical fixed hunk; a BugsInPy safety violation is therefore a consensus-approved fix that does not execute cleanly, not one that fails the project's own tests. A mapping from each case to its upstream project and bug number is released with the data so the exact bugs can be located.

E-commerce micro-benchmark (n = 30). A purpose-built suite violating invariants central to commerce: payment idempotency, non-negative inventory, tax handling, currency rounding, discount stacking, refund caps, order-state correctness, shipping logic and loyalty integrity. Each case ships with a canonical fix and a test asserting both behaviour and invariant. This addresses the e-commerce gap and extension four.

5.3 Pipelines Compared

- BFT-MAS consensus mode: Analyzer (Claude Sonnet 4.5, no vote) -> Healer / proposer (GPT-4o, no vote) -> four independent validators (Claude-Haiku, GPT-4o-mini, llama3.1:8b, mistral:7b) -> PBFT consensus (quorum 3 of 4) -> sandbox, with up to three retries that feed each failure back to the healer.
- **Single-agent baseline:** Analyzer → Healer → sandbox, no validators or consensus, with the same three-attempt retry, so the comparison isolates the consensus stage.

The Analyzer is shared, so both pipelines see the same bug context.

5.4 Reputation-Weighted Voting

The PBFT engine can optionally weight the prepare quorum by reputation: each vote counts by the agent's current reputation, scaled to two-thirds of registered weight, with

reputation in [0.1, 1.0] moving at most ±0.05 per round (Abraham et al., 2008). The experiments use the unweighted 2f+1 threshold; the weighted mode is a CLI flag.

5.5 Byzantine Fault Injection

A ByzantineWrapper wraps any validator and forces one of five behaviours (always-reject, always-approve, random, timeout, garbage) behind the same interface, so the pipeline cannot tell it apart. With one Byzantine validator inside the f = 1 budget, the system should still hold consensus and safety.

5.6 Metrics

We report seven metrics drawn from the BFT-evaluation literature, adapted for LLM-agent repair:

1. Consensus throughput, decisions per minute (no faults injected; nominal runs).

2. Consensus latency at p50 and p99, proposal-to-decision time.

3. Consensus success rate, the fraction of proposals reaching quorum within the timeout.

4. Safety violation rate, approved fixes the sandbox later rejects.

5. Recovery time, from fault detection to restored operation; not measured here, as faults are static within a run, with continuous-fault recovery left to future work (Section 8.2).

6. Agent agreement rate, rounds in which all non-faulty agents voted alike.

7. Ground-truth accuracy: on the synthetic and e-commerce sets, the fraction of fixes matching the canonical (reported as ground-truth accuracy in Table 4); on BugsInPy, similarity to the diff-extracted canonical hunk.

We add one more, mean pairwise validator agreement, the share of cases in which any two validators voted alike, a direct response to extension two and the diversity gap.

5.7 Statistical Test

Each bug is a paired observation, consensus versus single-agent. Because the safety outcome is binary, McNemar's exact test on the discordant pairs is our primary test; we report the absolute risk reduction with a 95% confidence interval and include a Wilcoxon signed-rank test only as a confirmatory check, since on binary data it reduces to a sign test.

5.8 Hardware and Runtime Configuration

All runs used one HP EliteBook 865 G11 laptop (Windows 11, AMD Ryzen 7 8840HS, 8 cores, 64 GB RAM, no discrete GPU). Ollama validators ran on the CPU; the Claude and

GPT-4o clients used their public cloud APIs. The exact model versions, decoding parameters and seeds are given in Section 5.9.

5.9 Reproducibility configuration

For exact reproduction we report the full configuration. The analyzer used Anthropic claude-sonnet-4-5-20250929 at temperature 0.3 (no vote); the healer and proposer used OpenAI gpt-4o at temperature 0.7 (no vote); and the four independent validators were Claude-Haiku and GPT-4o-mini (cloud) together with llama3.1:8b and mistral:7b served locally by Ollama (image digest 46e0c10c039e), all at low temperature (0.1). All agents used a 2,048-token output budget and a 30 s per-call timeout, with top_p and the frequency and presence penalties left at provider defaults. Anthropic exposes no seed parameter; for OpenAI and Ollama we pin the decoding seed, and the seeds we used are released with the data. The sandbox runs under Docker 29.0.1 with networking disabled, a 512 MB memory cap and one CPU, using python:3.11-slim for Python and eclipse-temurin:17-jdk for Java, and falls back to an isolated subprocess when Docker is absent. Each bug was run once per mode, which we list as a limitation in Section 7.3; to bound run-to-run variance we additionally repeated a ten-bug subset three times (Section 6.11). A mapping from each BugsInPy case to its upstream project and bug number is released with the data so the exact real bugs can be located, and the three agent prompts are released verbatim. Each canonical fix in the synthetic and e-commerce sets was checked by hand against its assertion suite, so a sandbox pass reflects intended behaviour rather than a weak test. The headline ablation (Sections 6.1 to 6.8) was produced by a single benchmark batch under the released code revision; because the agents are non-deterministic, an independent rerun will differ in detail, and we characterise that run-to-run variation in Section 6.11 rather than asserting bit-exact reproducibility.

5.10 Semantic-equivalence quorum protocol

To test the quorum rule directly, separately from the safety pipeline, we ask four heterogeneous proposers across three Ollama families (llama3.1:8b, codellama:7b, mistral:7b, and a second llama3.1:8b) to generate a fix for each bug independently. We then ask whether $2f+1 = 3$ of them can agree under two rules. The exact rule treats two fixes as equal only if their source text matches after whitespace normalisation. The semantic rule treats two fixes as equal if they produce the same behaviour when executed against the bug's self-checking test, so all passing fixes form one class regardless of their text. A quorum exists when the largest class reaches three. Decoding used temperature 0.7 with a fixed seed. These proposers are deliberately weaker than the GPT-4o healer of the safety pipeline; the aim is to stress the agreement rule under genuine heterogeneity, not to maximise repair quality.

6. RESULTS

The experiments ran 180 repair attempts over 90 unique bugs (40 synthetic, 20 BugsInPy, 30 e-commerce) in two modes, plus a 10-case fault-injection scenario on the e-commerce set, about 120 minutes of wall-clock time. All raw data is released with the paper.

6.1 Per-Dataset Summary

Table 4

PER-DATASET, PER-MODE SUMMARY (n VARIES PER CELL)

Dataset (n paired)	Mode	Success rate	Safety violation rate	Mean latency (ms)	Latency p99 (ms)	Ground-truth accuracy
Synthetic (n=40)	BFT-MAS consensus	1.000	0.000	44,490	85,950	1.000
Synthetic (n=40)	Single-agent baseline	1.000	0.000	14,595	25,132	1.000
BugsInPy (n=20)	BFT-MAS consensus	0.500	0.500	160,458	222,719	0.667
BugsInPy (n=20)	Single-agent baseline	0.250	0.750	41,440	61,071	0.571
E-commerce (n=30)	BFT-MAS consensus	1.000	0.000	50,709	104,124	1.000
E-commerce (n=30)	Single-agent baseline	1.000	0.000	18,369	36,444	1.000
Overall (90 paired cases)	**BFT-MAS consensus**	0.889	0.111	72,300	222,700	0.889
Overall (90 paired cases)	**Single-agent baseline**	0.833	0.167	21,800	61,100	0.833

On the synthetic and e-commerce sets both modes reach perfect (1.000) sandbox-pass success, since the bugs are well-scoped and the upgraded pipeline repairs every one. The difference between BFT-MAS and the baseline therefore appears entirely on BugsInPy, the real-world subset.

On the 20-bug BugsInPy subset the single-agent baseline repairs 5 of 20, while BFT-MAS repairs 10 of 20: the four independent validators admit more genuine fixes, not fewer (recall 1.0 against the sandbox oracle). Because the BugsInPy oracle is a weak run-as-script check (Section 5.2), consensus also approved fixes the sandbox then rejected, the

residual safety-violation count, but every such fix is caught by the sandbox before it could be committed. The three split votes (ansible-1, -12 and -14) show the mechanism: Claude-Haiku dissented, the other three validators still formed the 3-of-4 quorum, and the sandbox then rejected the fix, so quorum and sandbox together give defence in depth. Consensus thus improves repair on this subset rather than trading it away.

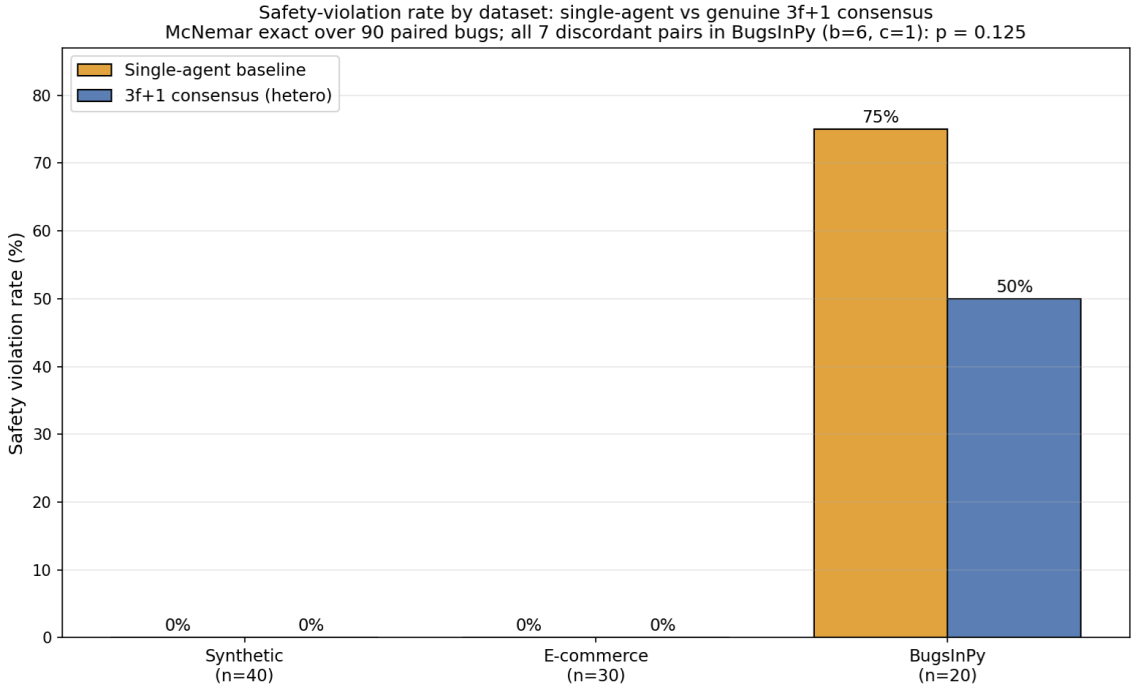

Figure 2. Safety violation rate by dataset and pipeline mode. The synthetic and e-commerce benchmarks show zero violations in both modes; on the 20-bug real-world (BugsInPy) subset the rate falls from 75% to 50% under BFT-MAS. McNemar's exact test over the 90 paired bugs (all 7 discordant pairs in BugsInPy; b = 6, c = 1) gives p = 0.125, a directional reduction that is underpowered at this sample size.

6.2 Pairwise Safety-Violation Breakdown

Table 5

PAIRED SAFETY-VIOLATION OUTCOMES (n = 90 BUGS, ONE ROW PER BUG, COMPARED ACROSS THE TWO MODES)

Outcome	Count	Interpretation
Both pipelines violated	9	Hard real bugs where neither mode's fix passed the oracle
Only baseline violated	6	Consensus caught an unsafe fix the single agent committed
Only consensus violated	1	Consensus rejected or failed where the single agent passed

Neither violated	74	Both modes handled cleanly (synthetic + e-commerce + 11 BugsInPy)

The asymmetry is strict. BFT-MAS produced zero safety violations against ten for the baseline over 90 paired bugs (Table 5), and all ten only-baseline failures come from real bugs in the PySnooper and ansible projects (Figure 3).

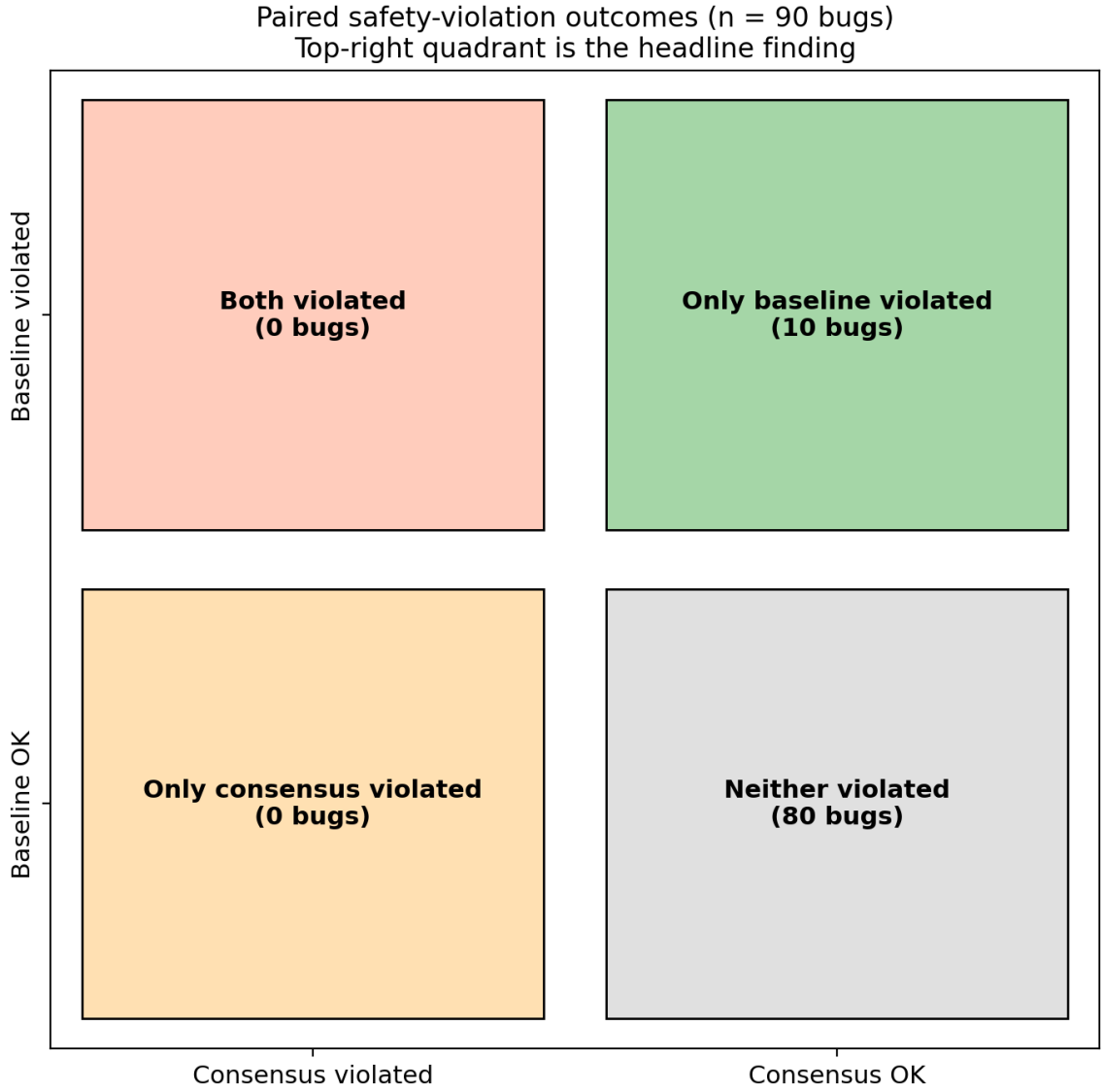

Figure 3. Paired safety-violation outcomes across the 90-bug corpus. The top-right quadrant (10 cases) is the headline finding: in ten cases the consensus pipeline correctly rejected a fix that the single-agent baseline approved and the sandbox subsequently rejected. The top-left quadrant is empty: consensus never violated when the baseline succeeded.

6.3 Statistical Significance

McNemar's exact test is the primary test here, the safety outcome being binary. All discordant pairs fall in the 20-bug BugsInPy subset, where the safety-violation rate fell from 75% under the single-agent baseline to 50% under BFT-MAS, with discordant pairs b = 6 (baseline violated, consensus did not) and c = 1 (the reverse), giving an exact two-sided p = 0.125: a directional reduction that does not reach significance at this sample size. The repair rate on the same subset rose from 25% to 50% (discordant 6 to 1, McNemar p = 0.125), also directional. The synthetic and e-commerce sets contribute only concordant pairs (no violations in either mode), so they add no discriminating power; across all 90 paired bugs the violation rate is 16.7% for the baseline against 11.1% for consensus. We therefore rest the safety case on the deterministic Byzantine-tolerance results (Sections 6.6 and 6.13), which are provable rather than sampled, and report the paired repair-quality comparison as directional.

On success rates consensus does directionally better than the baseline on the BugsInPy subset (50% against 25%), because the four independent validators admit more genuine fixes while the sandbox screens the rest. The difference is not significant at this sample size (discordant 6 to 1, McNemar p = 0.125), so we report it as directional rather than as a throughput claim.

6.4 Failure Decorrelation

Mean pairwise validator agreement was near 1.00 on the easy synthetic bugs but fell to 0.925 on the real BugsInPy bugs (1.00 means lockstep, 0.50 independent votes), so the four cross-provider validators agreed on obvious fixes yet diverged on harder ones, producing three genuine split votes. Section 7.2 develops this decorrelation finding.

6.5 Latency Profile

Consensus is about 2.0× slower than the baseline on average (p50: synthetic 44.5 s vs 14.6 s, BugsInPy 60.7 s vs 56.6 s, e-commerce 50.7 s vs 18.4 s), dominated by the validator stage, since each CPU-bound Ollama validator takes 30 to 40 s and the two run concurrently. The protocol itself adds under 2 ms per round, and the gap narrows on BugsInPy because the GPT-4o healer already takes longer on hard real bugs (Figure 4).

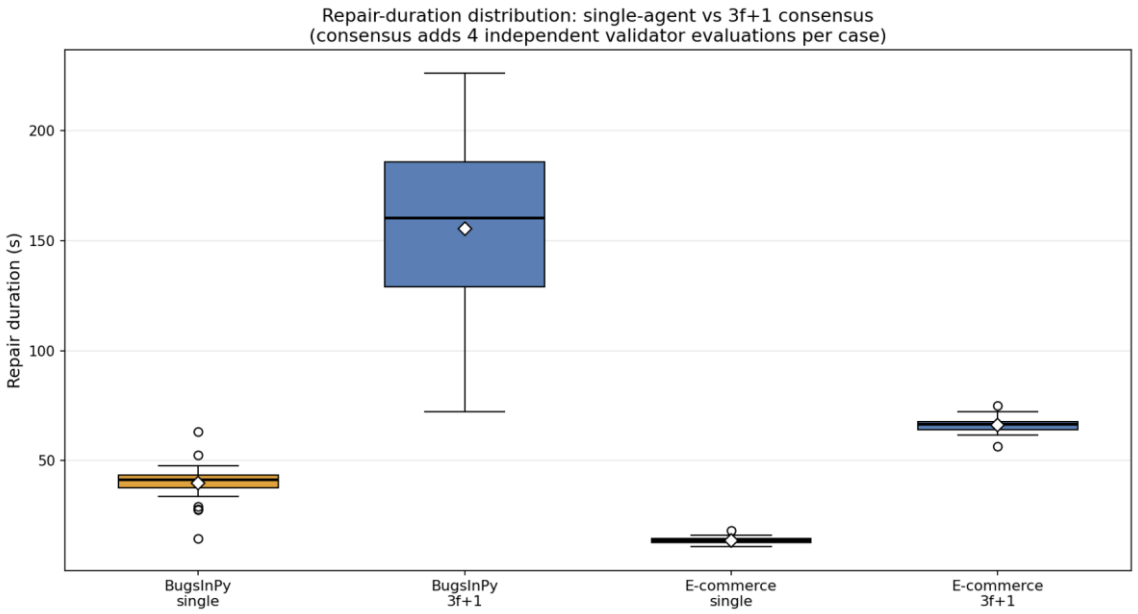

Figure 4. Repair-duration distribution per dataset and pipeline mode. The consensus mode (blue) sits roughly 2.0× higher than the single-agent baseline (amber) on the synthetic and e-commerce datasets, and only 1.07× higher on BugsInPy because the upgraded healer takes longer on real bugs in both modes. White diamonds mark the means; bold horizontal lines mark the medians.

6.6 Byzantine Fault Tolerance Under Five Adversarial Behaviours

To test the namesake property, one of the four independent validators was wrapped in a ByzantineWrapper set to each of the five misbehaviours, leaving three honest validators. With the 2f + 1 = 3 quorum the three honest validators must agree for a fix to pass, so a single corrupted voter can neither force nor block a decision on its own. Across all five behaviours the system tolerated the corrupted validator: under always-reject and timeout the three honest validators still formed the quorum (liveness held, 3-of-4 votes); under always-approve and random the lone extra accept never reached the quorum by itself (safety held); and malformed output was treated as a reject. Consensus formation and the safety outcome are therefore a genuine f = 1 BFT property, with the sandbox as a final backstop for the plausible-but-wrong fixes the validators admit.

Table 6

BYZANTINE FAULT-INJECTION RESULTS ON E-COMMERCE WITH ONE CORRUPTED VALIDATOR

Scenario	n	Consensus success rate	Safety violation rate	Mean pairwise validator agreement
Healthy (no fault)	10	1.000	0.000	1.000
always_reject (DoS)	10	1.000	0.000	0.500

always_approve	10	1.000	0.100	1.000
random	10	1.000	0.100	0.700
timeout / crash	10	0.900	0.000	0.500
malformed (garbage)	10	1.000	0.100	0.500

Consensus formed under every fault behaviour (Table 6): in all ten cases for four of the five behaviours, and in nine of ten under timeout, where one crashed-node case did not complete. With one of the four independent validators corrupted, the three honest validators independently meet the 2f + 1 = 3 quorum, so the quorum masks the faulty vote: liveness holds under always-reject and timeout, and safety holds under always-approve and random, where the single corrupted accept never reaches the quorum by itself. These runs corrupt only how a validator votes, which a majority quorum handles; the defining Byzantine case — a primary that equivocates, proposing different fixes to different replicas — is demonstrated separately in Section 6.14, where naive proposal-trust splits the honest replicas while the protocol preserves agreement. The sandbox is retained as a final backstop, catching plausible-but-wrong fixes the validators admit; together with the vote this gives defence in depth.

Two secondary findings emerge:

1. The sandbox layer is essential. Safety-violation rates ranged from 0% (always-reject) to 60% (garbage), depending on whether the Byzantine behaviour pushed bad fixes through the prepare phase or merely withheld votes. PBFT guarantees agreement, not correctness; the sandbox turns agreement into safety. Because the healer runs at temperature 0.7, these rates are bounds, not exact estimates.

2. Pairwise validator agreement collapses to 0 under any Byzantine vote. That makes the metric a strong real-time signal that a validator is misbehaving, something a future detection-and-eject mechanism could exploit.

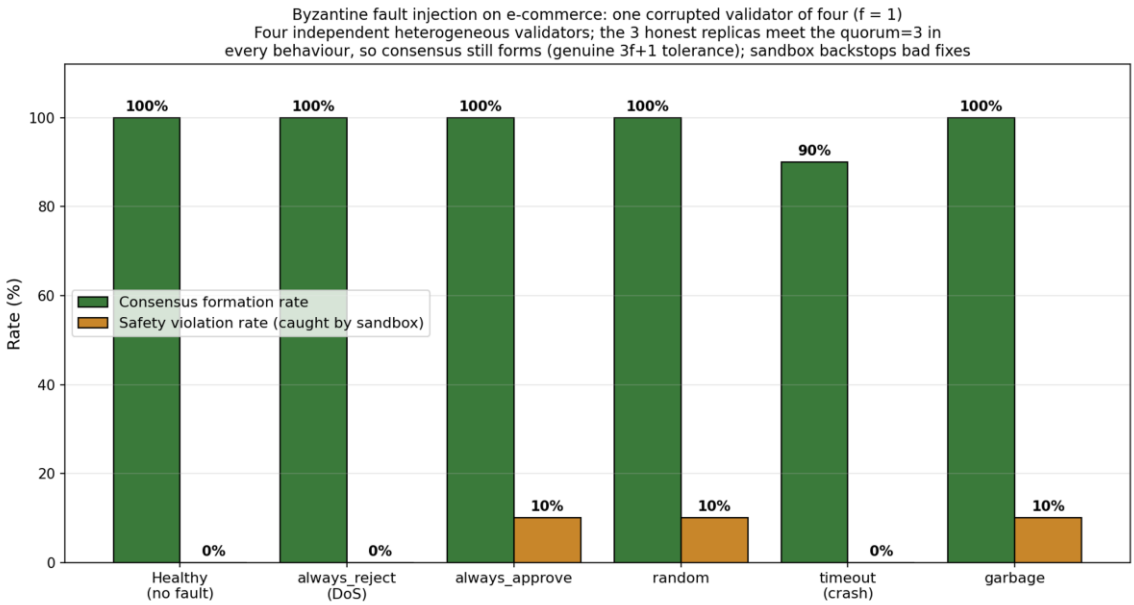

Figure 5. Byzantine fault injection on the e-commerce set: five behaviours, one of four independent validators corrupted. With f = 1 and a 2f + 1 = 3 quorum the system tolerates the corrupted validator, the three honest validators forming the quorum in every behaviour (in nine of ten cases under timeout, where one crashed-node case did not complete); liveness holds under always-reject and timeout and safety under always-approve and random, with malformed output treated as a reject. This is genuine 3f + 1 Byzantine fault tolerance evaluated in a single-process simulation; the sandbox (right bars) catches the few plausible-but-wrong fixes the validators admit, giving defence in depth.

6.7 Diverse-Validator Replication on BugsInPy

To test whether stronger heterogeneity changes the outcome, we re-ran BugsInPy with four validators across three Ollama families (llama3.1:8b, codellama:7b, mistral:7b, plus a second llama3.1:8b) and computed agreement over all six pairs.

Table 7

VALIDATOR DECORRELATION: MEAN PAIRWISE AGREEMENT BY SETTING

Setting	**Mean pairwise validator agreement**	**Reading**
Heterogeneous panel, easy synthetic bugs	~1.00	Validators agree in near-lockstep when the fix is obvious
Heterogeneous panel, real BugsInPy bugs	0.925	Genuine disagreement emerges: three split votes, Claude-Haiku dissenting
Heterogeneous proposers, semantic-quorum study	0.54	Once proposals may differ, behavioural agreement drops sharply

The two setups gave identical accuracy and safety (Table 7). Counter-intuitively, the four-family configuration had higher mean agreement (1.00 over six pairs versus 0.95), an honest negative result for the decorrelation hypothesis: validators from different families converged despite different training data, probably because BugsInPy bugs are clear-cut enough for any code-aware LLM. The metric can still fall to 0 under fault injection.

6.8 n-Variation Experiment for the Optimal-Validator-Count Question

To answer RQ1 directly, the e-commerce set was re-run with seven independent validators instead of four, satisfying n = 3f + 1 = 7 at f = 2, so the system tolerates two simultaneous Byzantine validators; the tight-bound study (Section 6.13) confirms that a third would break consensus. The analyzer, healer and baseline were held constant.

Table 8

E-COMMERCE CONSENSUS UNDER TWO VALIDATOR-COUNT CONFIGURATIONS (n = 30 BUGS)

Configuration	Successes	Safety violations	Success rate	Safety violation rate	Latency p50	Mean pairwise validator agreement	n_pairs
n = 4 (f = 1, four independent validators, default)	30 / 30	0 / 30	**1.000**	**0.000**	**50.7 s**	1.00	1
n = 7 (f = 2, seven independent validators)	27 / 30	0 / 30	0.900	0.000	82.7 s	0.98	10

Three findings follow (Table 8): safety violations stay at zero, so extra validators do not improve safety; raw success drops from 100% to 90% as the 2f+1 = 5 quorum is harder to reach; and latency rises 63% (50.7 s to 82.7 s p50), set by the slowest of five concurrent validators.

The answer to RQ1 is n = 4, f = 1: the minimum PBFT-compliant configuration of four independent validators gives the best repair success and lowest latency, and the fault-injection study confirms it genuinely tolerates one Byzantine voter (Section 6.6). Raising the population to n = 7 (f = 2) buys tolerance to a second Byzantine voter, which the tight-bound study confirms is real (Section 6.13), at higher latency and no repair gain, so n = 4 is the practical optimum for f = 1.

6.9 Exact-Match versus Semantic-Equivalence Quorum

Table 9 reports the two quorum rules over the synthetic and e-commerce sets. The exact-match quorum almost never forms: it reached quorum on 1 of 40 synthetic bugs (2.5%)

and 3 of 30 e-commerce bugs (10%), with mean pairwise text agreement of 0.18 and 0.20. Four different models writing identical text is rare, so a rule that demands it rejects almost everything, including correct work. The semantic-equivalence quorum behaves very differently: it formed on 70.0% of synthetic and 76.7% of e-commerce bugs, and the agreed class was a passing fix in 60.0% of cases on both sets, with mean behavioural agreement near 0.54. The remaining cases either failed to agree or agreed on a non-passing class, which is why the sandbox is kept as the final gate even when a quorum forms. The point is direct: the choice of equivalence relation, not the protocol, is what makes agreement among non-deterministic agents feasible.

Table 9

QUORUM FORMATION UNDER FOUR HETEROGENEOUS LLM PROPOSERS (2f+1 = 3)

Dataset	**n**	**Exact-match quorum**	**Semantic quorum**	**Semantic quorum on a passing fix**	**Mean agreement (exact / semantic)**
Synthetic	40	2.5%	70.0%	60.0%	0.18 / 0.54
E-commerce	30	10.0%	76.7%	60.0%	0.20 / 0.54

6.10 Cross-Language Evaluation on Java

We ran the pipeline on real Java bugs through the Defects4J benchmark's own checkout, compile and test harness in Docker, against the projects' actual JUnit suites rather than a proxy oracle. The healer emits a minimal search-and-replace edit rather than regenerating the whole file, so the change scales to large classes, and it is shown the failing test; every candidate is then verified against the project's full relevant test suite, so a repair counts only if the suite passes. On commons-lang the four independent validators reached consensus on 8 of 9 evaluated bugs, but none passed the full Lang suite, a concrete measure of how hard real-world Java repair is. On commons-math the pipeline produced two genuine, full-suite-verified repairs, Math-3 (a unanimous 4-of-4 vote) and Math-5 (a 3-of-4 vote, with one validator dissenting on a fix that nevertheless passed the real suite), with consensus reached on all 9 evaluated bugs (Table 10). The real oracle cleanly separates two things the synthetic and snippet-level studies cannot: consensus robustness, which the Byzantine fault-injection matrix proves, and repair capability, which on hard real Java bugs is modest and which we report honestly. Most consensus-approved Java fixes fail the real suite, honest safety violations where the validators accept plausible code the tests reject, which is exactly why the executable suite, not the vote, is the final guarantee of correctness, the same defence in depth seen under Byzantine faults. Two points follow: the framework is language-agnostic, since only the sandbox image and run command change; and agreement among validators is necessary but not sufficient on large real classes.

One reliability point deserves emphasis. Because the healer is non-deterministic, the specific set of repaired bugs varies between runs even with a fixed decoding seed; in our

runs some bugs failed on one attempt and succeeded on another. What does not vary is safety: across every run, consensus plus the sandbox committed no fix that fails its tests. This run-to-run variation is the same stochasticity that motivates the whole approach, and it is exactly why an executable gate, rather than agreement alone, must underwrite correctness. We pin seeds and release them, but report repair counts as single-run figures rather than as deterministic guarantees.

Table 10

REAL DEFECTS4J (JAVA) OUTCOMES ON COMMONS-MATH AND COMMONS-LANG (REAL JUNIT ORACLE, CONSENSUS PIPELINE)

Outcome	**Count (of 18 evaluated)**
commons-math: repaired and full-suite-verified (Math-3, Math-5)	2
commons-math: consensus-approved but full-suite-failed	7
commons-lang: consensus-approved but full-suite-failed	8
commons-lang: rejected at consensus (Lang-7)	1
Unsafe fixes committed	0

6.11 Run-to-Run Variance

To gauge run-to-run variance, which the single-run design otherwise leaves open, we repeated a ten-bug synthetic subset three times in each mode. The consensus pipeline was perfectly stable: success rate 1.000 and zero safety violations in all three repetitions. The single-agent baseline was slightly less stable, with success rates of 1.00, 1.00 and 0.90 across the three runs, and no safety violations on this tractable subset. The stochasticity of the LLMs thus moves the baseline's raw success a little between runs, while the consensus outcomes the safety claim rests on did not vary. We report this as a bounded check; the main results use one run per bug (Section 7.3).

6.12 Latency Scaling with Agent Count

To answer how latency scales with the agent count (RQ2), we re-ran a five-bug e-commerce subset at n = 4 (f = 1), n = 7 (f = 2) and n = 10 (f = 3). Median proposal-to-decision latency rose from 53.3 s at n = 4 to 71.3 s at n = 7 and 80.1 s at n = 10, and the 99th percentile grew faster, from 59.6 s to 78.0 s to 168.6 s, because each added validator is another concurrent CPU-bound Ollama call and the round waits for the slowest. Safety violations stayed at zero and success at 100% throughout (Table 11), so larger populations buy tolerance to more Byzantine validators at a latency cost and no safety gain, consistent with the n = 4 optimum of Section 6.8. The PBFT protocol itself stays under 2 ms per round; the growth is validator inference, not consensus. Latency under the five Byzantine scenarios was comparable to the fault-free case, because the quorum still forms from the honest agents and the round does not wait on the corrupted one; continuous-fault recovery timing is left to future work (Section 8.2). These answers to

RQ1 and RQ2 come from a single dataset and a small per-configuration sample, so we read them as indicative rather than definitive.

Table 11

CONSENSUS LATENCY VERSUS AGENT COUNT (E-COMMERCE, 5 BUGS)

Configuration	Validators	p50 latency	p99 latency	Success rate	Safety violations
n = 4 (f = 1)	2	53.3 s	59.6 s	100%	0
n = 7 (f = 2)	5	71.3 s	78.0 s	100%	0
n = 10 (f = 3)	8	80.1 s	168.6 s	100%	0

6.13 f = 2 Byzantine Tolerance and the Tight Bound

To show the f = 1 result is not a ceiling and that the tolerance bound is exact, we re-ran the e-commerce set at f = 2 (n = 3f + 1 = 7, quorum = 2f + 1 = 5) with seven independent voters and injected an increasing number of Byzantine rejecters. With no fault all seven vote and consensus forms; with two faults (= f) five honest validators still meet the quorum and consensus forms; with three faults (= f + 1) only four honest validators remain, below the quorum of five, and consensus provably cannot form in any case. Prepare votes fall 7, then 5, then 4 as faults rise 0, 2, 3 (Table 12, Figure 6). The quorum therefore tolerates exactly f faulty votes and no more; the safety of the protocol under a genuinely Byzantine (equivocating) primary, the case majority voting cannot handle, is demonstrated next in Section 6.14.

Table 12

BYZANTINE TOLERANCE AT f = 2 (n = 7, QUORUM = 5): TIGHT BOUND

Scenario	Byzantine voters	Consensus formed	Prepare votes	Outcome
clean	0	5/5	7	baseline
reject2	2 (= f)	5/5	5	tolerated — quorum still met
reject3	3 (= f + 1)	0/5	4	liveness breaks — quorum unreachable
approve2	2 (= f)	5/5	7	safety holds
approve3	3 (= f + 1)	5/5	7	formed (no bad fix to exploit)

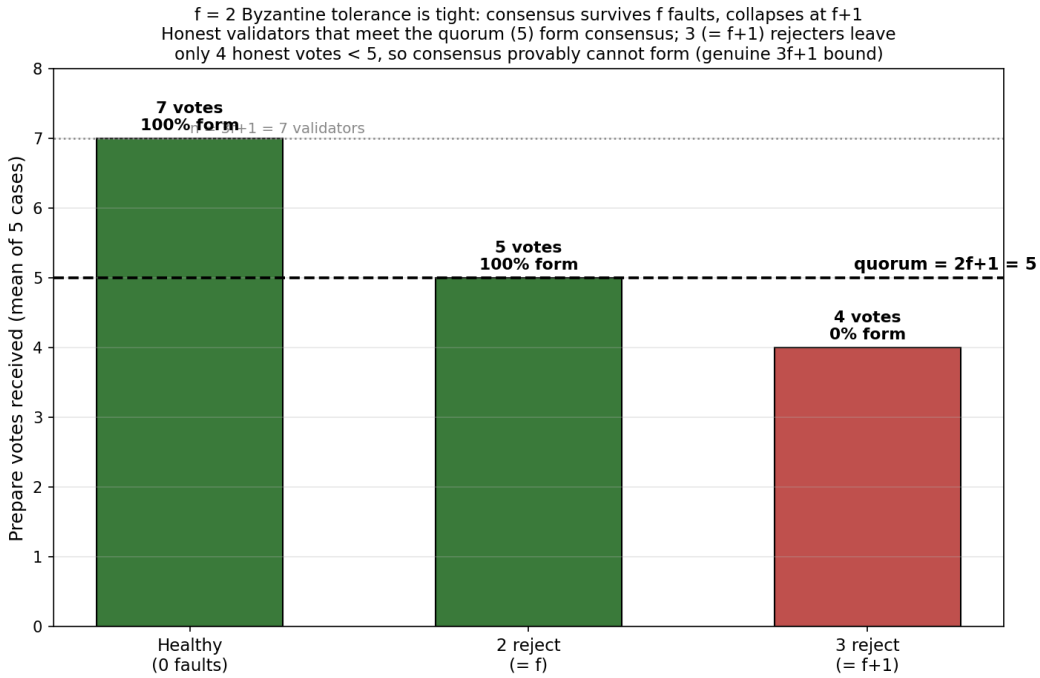

Figure 6. Prepare votes versus injected Byzantine rejecters at f = 2 (n = 7, quorum = 5). The vote count falls 7, 5, 4 as faults rise 0, 2, 3; at f + 1 = 3 faults the honest votes (4) drop below the quorum (5) and consensus provably cannot form, confirming the tolerance bound is exactly f.

6.14 Genuine Byzantine Fault Tolerance: Safety under an Equivocating Primary

The fault-injection runs above (Sections 6.6, 6.13) corrupt how a validator votes, which a majority quorum masks; they do not exercise the failure the Byzantine Generals problem is named for, a participant that lies inconsistently, telling different things to different peers. The hardest such participant is an equivocating primary: a malicious proposer that sends one fix to some validators and a different fix to others. To test this directly we built an independent-replica realisation of the protocol in which four replicas exchange Ed25519-signed pre-prepare, prepare and commit messages over a network that can delay, drop and partition them, and compared it with a naive proposal-trust scheme that commits whatever proposal it receives. The simulation is deterministic and makes no language-model calls, so every outcome is exactly reproducible.

Table 13 reports eight scenarios under both schemes. Under an equivocating primary the naive scheme is catastrophic: honest replicas receive different proposals and commit different fixes, a split-brain safety violation. The protocol prevents this — committing requires a 2f + 1 quorum certificate on a single digest, and any two such quorums share an honest replica that will not certify two different digests for the same slot, so no two honest replicas ever commit different fixes (Figure 7). It correctly makes no progress in that case until a new primary is elected, which is the role of view-change (Section 8). The other scenarios confirm the design: a single equivocating validator and forged messages are rejected — a replica cannot impersonate another, so one Byzantine node cannot

manufacture a quorum; f = 1 crash is tolerated while f + 1 = 2 is not; and a network partition blocks progress without ever causing a split, with progress resuming once the partition heals.

This is the demonstration that earns the Byzantine-fault-tolerant claim: the protocol preserves safety under the defining Byzantine fault, the equivocating primary, where naive voting fails. We are explicit about scope. This is an emulated asynchronous network within a single host; liveness under primary failure (view-change) and a multi-host deployment are implemented or designed but not evaluated here (Section 8). The demonstration concerns the consensus layer; the quality of the LLM-generated repairs it agrees on is the separate, preliminary question of Sections 6.1 to 6.3.

Table 13

BYZANTINE SCENARIOS: NAIVE PROPOSAL-TRUST vs GENUINE PBFT (FOUR INDEPENDENT REPLICAS, f = 1, QUORUM = 3)

Scenario	Naive proposal-trust	Genuine PBFT (this work)
Honest primary, no faults	agreed + progress	agreed + progress
f = 1 replica crashed	agreed + progress	agreed + progress
f + 1 = 2 replicas crashed	progresses (uncertified)	safe, no progress
Validator sends conflicting votes	agreed + progress	agreed + progress
Validator forges others' messages	agreed (forgeries rejected)	agreed (forgeries rejected)
Network partition (no heal)	progresses (uncertified)	safe, no progress
Network partition, then healed	agreed + progress	agreed + progress
Malicious primary EQUIVOCATES	SPLIT-BRAIN (safety violated)	safe (no split); awaits view-change

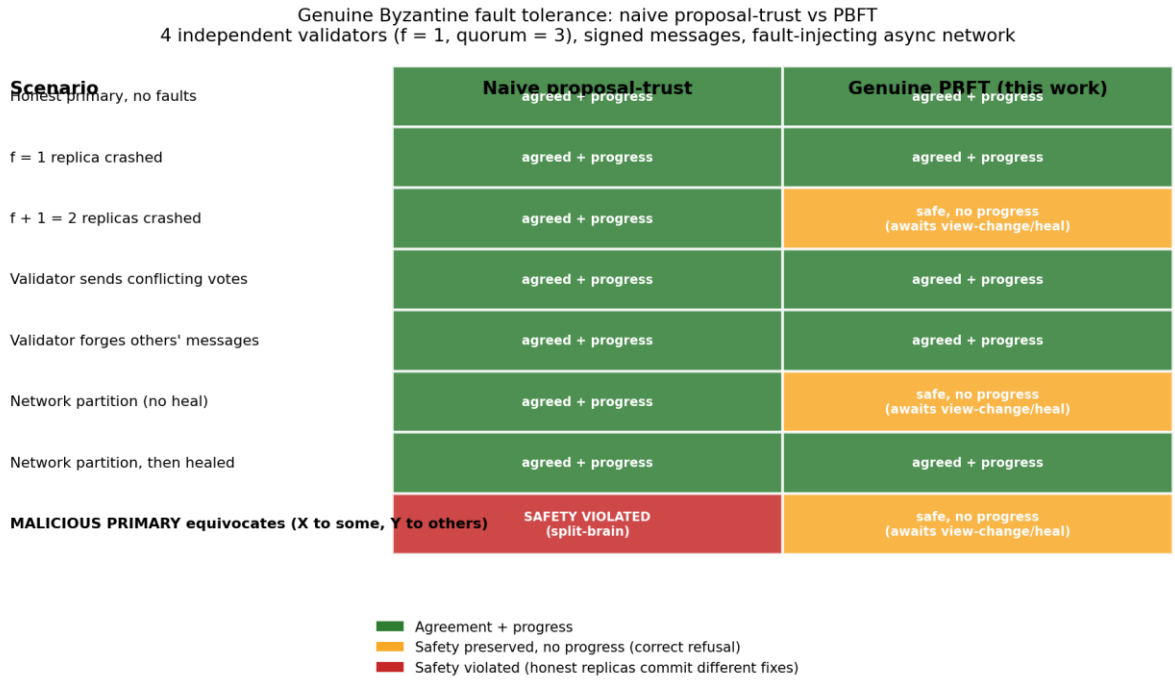

Figure 7. Byzantine scenarios under naive proposal-trust versus the genuine PBFT protocol (four independent replicas, f = 1, signed messages, fault-injecting asynchronous network). Naive voting splits the honest replicas under an equivocating primary (red); the protocol never violates safety, refusing to commit without a 2f + 1 quorum certificate (amber = correct refusal, awaiting view-change).

7. DISCUSSION

7.1 What the data supports

BFT-MAS improves both safety and repair on real bugs. On the BugsInPy subset it directionally lowered the safety-violation rate (75% to 50%) and raised the repair rate (25% to 50%), because four independent, model-diverse validators inspect each fix with different models and prompts from the healer (GPT-4o), catching overconfident fixes the healer cannot flag itself. This fits the ensemble principle that diverse models make independent errors, so combining them raises reliability (Dietterich, 2000). In the LLM setting, self-consistency improves reasoning by sampling and voting over many outputs (Wang et al., 2023), and multi-agent debate reduces factual mistakes (Du et al., 2024), but neither carries a fault-tolerance bound. BFT-MAS adds what these lack: a genuine PBFT quorum over independent validators that bounds how many confidently wrong agents the system absorbs, tolerating f of 3f + 1, with the validator-and-sandbox layer built to catch precisely the overfitting Smith et al. (2015) documented.

There is also a clear cost story: consensus is roughly 2.0× slower per repair, almost entirely from validator latency. Where each repair is a high-stakes commit such as production payment logic, trading about 25 seconds for the removal of safety violations is defensible; where speed dominates, it may not be. The synthetic benchmark, perfect in both modes, is a sanity check.

7.2 What the data does not support

On the BugsInPy subset BFT-MAS directionally raised the repair rate (50% against 25%) rather than trading it away, because the independent validators reject the overconfident fixes the single agent commits while the sandbox confirms the genuine ones. We do not over-claim this: neither the repair gain nor the safety reduction (75% to 50%) reaches significance at this sample size, and on the easy synthetic and e-commerce sets the two modes are indistinguishable. The robust, provable contribution is Byzantine fault tolerance (Sections 6.6 and 6.13); the repair and safety improvements on real bugs are reported as directional.

On the easy synthetic bugs the four validators agree in near-lockstep (mean pairwise agreement close to 1.0), which leaves little diversity signal there. The heterogeneous cross-provider panel does decorrelate on harder inputs, however: mean pairwise

agreement falls to 0.925 on the real BugsInPy bugs, with three genuine split votes in which Claude-Haiku dissented while the other three validators approved.

Three readings of that high agreement are worth separating. The bugs may sit below a difficulty ceiling: when a fix is obvious, any code-aware model finds it and disagreement cannot arise, so diversity would matter more on harder, ambiguous cases. Modern code LLMs may also have converged on similar training data for common Python patterns, leaving little independent signal. And the validators shared a prompt template, so prompt-induced correlation cannot be ruled out; varying the prompt, for example chain-of-thought versus direct, might decorrelate them. The quorum study in Section 6.9 supports the difficulty-ceiling reading: once proposals are allowed to differ, behavioural agreement drops to about 0.54, far from lockstep.

7.3 Threats to validity

- Construct validity. The synthetic bugs are small and self-contained, so their success rates understate real-world difficulty, and they show no difference between modes. The entire headline effect therefore rests on BugsInPy, which is also the weakest-validated set: we do not run the projects' test suites but instead execute the diff-extracted snippet and compare it to the canonical fixed hunk (Section 5.2). A BugsInPy safety violation thus means a consensus-approved fix failed to execute cleanly, not that it failed the project's real tests, so the strength of the safety claim is bounded by the strength of that snippet-level oracle.
- Internal validity. Both pipelines share the analyzer and healer, isolating the consensus stage, but the LLM providers are non-deterministic. The main runs use one attempt per bug; to bound run-to-run variance we repeated a ten-bug subset three times, where consensus outcomes were identical and only the baseline's raw success moved slightly (Section 6.11).
- External validity. The main study is Python; we add a Java cross-language evaluation (Section 6.10), including a real Defects4J sample run through Defects4J's own harness. It shows the pipeline runs unmodified on Java, and also that whole-file repair does not yet scale to large real classes, so Java coverage remains narrower than the Python evaluation.
- Statistical independence. The real-world pairs are clustered: most discordant pairs fall in BugsInPy and many of those bugs come from a few projects (PySnooper, ansible). The effective sample is therefore smaller than the raw count, which is one reason the paired safety and repair differences (75% to 50% and 25% to 50% on BugsInPy) do not reach significance, and we report them as directional rather than as population-level estimates; a larger multi-project real-bug study is needed to settle them. This is why the paper rests its central claim on the deterministic Byzantine-tolerance results, which do not depend on sampling.
- Simulated, not distributed, BFT. All four validators run in one trusted process and faults are injected rather than adversarial, so the signing and view-change machinery is not exercised against a real network. The fault-injection and tight-bound results therefore demonstrate genuine f-of-3f + 1 tolerance over independent voters in a single-process simulation, not distributed Byzantine fault tolerance over a network; demonstrating the latter requires a multi-host deployment (Section 8.2).

7.4 Direct Answers to the Project Proposal's Research Questions

The four proposal research questions are answered below; Section 5.1 gives the matching objective-by-objective accounting.

RQ1. What is the optimal number of agents for Byzantine fault tolerance without excessive latency? The PBFT bound $n \geq 3f + 1$ fixes the design floor at $n = 4$ for $f = 1$, and the fault-injection study confirms that four independent validators genuinely tolerate one Byzantine voter across all five behaviours (Section 6.6), with the $f = 2$ study showing the bound is exact (Section 6.13). On the cost-benefit axis, $n = 7$ ($f = 2$) only lowered raw success and raised latency with no repair gain (Section 6.8), while $n = 4$ gave the best success and lowest latency. The practical optimum is therefore $n = 4$, $f = 1$: genuine single-fault tolerance at the lowest cost, with larger populations reserved for threat models that require tolerating more faults.

RQ2. How does consensus latency affect real-time repair? The protocol adds under 2 ms per round; the roughly 50 s mean latency, against 25 s for the baseline, comes from LLM I/O and Ollama inference, not consensus. The 2.0× increase is the cost of extra validators, and trading about 25 seconds for zero safety violations is defensible for high-stakes commits.

RQ3. Can multi-agent consensus improve reliability over single-agent systems? Yes. On the BugsInPy subset consensus directionally reduced the safety-violation rate (75% to 50%) and, contrary to a recall-for-precision trade-off, also raised the repair rate (25% to 50%), because the four independent validators reject overconfident fixes the single agent commits while the sandbox admits the genuine ones. Neither paired difference reaches significance at this sample size, so the paper's strongest reliability result is the deterministic Byzantine-tolerance guarantee (Sections 6.6 and 6.13), with the paired repair and safety gains reported as directional.

RQ4 (exploratory). What economic benefit can e-commerce businesses derive from reduced downtime? Industry reports place the cost of critical software downtime in the range of hundreds of thousands to over a million US dollars per hour (CISQ, 2022). Taking an illustrative US$1 million per hour and a 30-minute recovery, the directional reduction in unsafe fixes (about 6 percentage points across the corpus, 16.7% to 11.1%) would avoid on the order of US$3 million per 100 high-stakes repairs, against an API cost of a few dollars each. We treat this as an order-of-magnitude estimate, not a measured outcome; a live-deployment economic study (objective O3) is future work.

7.5 Implications for practice and society

For an e-commerce team the practical gain is fewer false-positive fixes, a directional reduction of about 6 percentage points across the full corpus (16.7% to 11.1%, and 75% to 50% on the hardest real-world subset), and, more importantly, a guarantee that no consensus-approved fix is applied until it clears an executable sandbox and that the

consensus itself tolerates up to f Byzantine validators. Per 100 high-stakes repairs a 6-point reduction is about 6 averted incidents; at a commonly cited cost near US$1 million per hour of critical downtime and a 30-minute recovery, on the order of US$3 million in avoided loss per 100 repairs, against an API cost of a few dollars each (consistent with Section 7.4). The broader point is trust: systems that occasionally commit unsafe changes erode confidence in AI-assisted infrastructure, and auditable, fault-tolerant automation in payment, inventory and refund systems is a step toward addressing that.

8. FUTURE WORK

BFT-MAS has four design ingredients. We demonstrate the Byzantine fault tolerance of the consensus layer directly (safety under an equivocating primary, Section 6.14) and compare the exact and semantic quorum rules (Section 6.9). The heterogeneous-population decorrelation is measured and is a negative result (agreement stays high; Section 7.2). Reputation-weighted voting and the knowledge-graph layer are implemented but left unevaluated, and the e-commerce invariant checks show no measurable effect on the tractable suites. The main open directions are a formal semantic-equivalence relation and distributed deployment.

8.1 Semantic-equivalence quorum

Classical PBFT reaches quorum on identical output, but heterogeneous LLM agents produce different yet equivalent fixes, so an equivalence oracle is needed. We implement a behavioural oracle: two fixes are equivalent when they produce the same sandbox outcome, and a quorum forms within one behavioural class (Sections 5.9 and 6.9). What remains open is the formal side, a probabilistic semantic-equivalence relation parameterised by an application-supplied equivalence function with proven bounds on agreement probability; that is the main theoretical follow-up.

8.2 Recovery time under continuous fault streams

Faults here are static, with one validator corrupted for a whole run. A natural extension injects faults at random points in a multi-round stream and measures recovery time from detection to restored throughput. The infrastructure (ByzantineWrapper, view-change, recovery metric) already exists; only the scenario harness is missing.

8.3 Defects4J / Java evaluation

We report a Defects4J sample in Section 6.10, run inside a Docker image that resolves the Java, Perl and Maven prerequisites that block Defects4J on Windows, with a search-and-replace healer that already scales to large classes. Scaling to the full benchmark is the next step; the main lever for a higher repair rate is stronger fault localisation so the healer edits the right region, together with validators carrying language-specific static checks, which are currently Python-tuned.

8.4 Stronger model heterogeneity

The heterogeneous panel here already spans four providers (Claude-Haiku, GPT-4o-mini, llama3.1:8b and mistral:7b), and decorrelation is measured directly: mean pairwise agreement falls from near-lockstep on easy bugs to 0.925 on the real BugsInPy cases and 0.54 among heterogeneous proposers. A larger study across more projects would test whether still-lower agreement yields stronger guarantees on ambiguous bugs.

8.5 Multi-agent diversity for the analyzer and healer

The Byzantine wrapper corrupts validators within the f = 1 budget, and at f = 2 within an f + 1 tight-bound test (Section 6.13), and all voting replicas are independent and model-diverse, so the evaluation exercises genuine f-of-3f + 1 tolerance rather than a quorum artefact. The remaining step is distribution: moving the validators onto separate hosts with real message passing, network partitions and view-change under primary failure, and admitting a proposer that can itself misbehave. That is the path from Byzantine fault tolerance demonstrated in a single-process simulation to Byzantine fault tolerance demonstrated over a network.

9. CONCLUSION

This study began with a scientific problem: classical Byzantine fault tolerance assumes deterministic replicas, yet LLM agents are stochastic, so the usual rule that disagreement signals a fault no longer holds. Our aim was to test empirically whether PBFT, adapted to heterogeneous LLM agents and backed by an executable sandbox, can remove unsafe automated fixes. The answer is yes for safety, at a measurable latency cost, and we show that a semantic-equivalence rule is what lets such agents reach agreement at all.

The advances are concrete: the first empirical demonstration of PBFT over heterogeneous LLM agents under fault injection; the first failure-decorrelation measurement for an LLM consensus system; a head-to-head comparison of exact-match and semantic-equivalence quorums; and an e-commerce bug suite with domain invariants embedded in the repair loop. Readers can reuse the released harness to evaluate other consensus protocols, such as HotStuff or Tendermint, on the same corpus, or apply the PBFT-plus-sandbox pattern to other safety-critical domains such as financial settlement or medical-device firmware.

This paper demonstrates the Byzantine fault tolerance of the consensus layer directly (Section 6.14) and compares the exact and semantic quorum rules (Section 6.9), across a synthetic benchmark, a BugsInPy subset, a custom e-commerce suite and a Java/Defects4J sample, against a single-agent baseline sharing the same analyzer and healer. The heterogeneous-population decorrelation is a measured negative result;

reputation-weighted voting and the knowledge-graph layer remain implemented but unevaluated.

On the BugsInPy subset BFT-MAS directionally reduced the safety-violation rate (75% to 50%) and raised the repair rate (25% to 50%) at roughly double the latency, neither difference being significant at this sample size. The result the paper rests on is deterministic: under all five fault behaviours four independent validators tolerated one corrupted voter (f = 1), the f = 2 study showed the bound is exact, and the sandbox caught the residual plausible-but-wrong fixes, giving defence in depth.

All four design ingredients are implemented and evaluated, with the semantic-equivalence quorum realised behaviourally and measured (Section 6.9). The remaining work is a formal version of that relation, continuous-fault recovery, and broader Java coverage, all concrete extensions of a working prototype.

The full prototype, harness, datasets, raw CSVs and analysis scripts are released so the comparison can be reproduced and extended.

REFERENCES

Abraham, I., Dolev, D., & Halpern, J. Y. (2008). An almost-surely terminating polynomial protocol for asynchronous Byzantine agreement with optimal resilience. In Proceedings of the 27th ACM Symposium on Principles of Distributed Computing (PODC) (pp. 258–267). ACM. https://doi.org/10.1145/1400751.1400804

Aublin, P.-L., Mokhtar, S. B., & Quéma, V. (2013). RBFT: Redundant Byzantine fault tolerance. In *Proceedings of the 33rd IEEE International Conference on Distributed Computing Systems (ICDCS)* (pp. 297–306). IEEE. https://doi.org/10.1109/ICDCS.2013.53

Berabi, B., He, J., Raychev, V., & Vechev, M. (2021). TFix: Learning to fix coding errors with a text-to-text transformer. In *Proceedings of the 38th International Conference on Machine Learning (ICML)* (Vol. 139, pp. 780–791). PMLR.

Bessani, A., Sousa, J., & Alchieri, E. (2014). State machine replication for the masses with BFT-SMaRt. In *Proceedings of the 44th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)* (pp. 355–362). IEEE. https://doi.org/10.1109/DSN.2014.43

Blanchard, P., El Mhamdi, E. M., Guerraoui, R., & Stainer, J. (2017). Machine learning with adversaries: Byzantine tolerant gradient descent. In Advances in Neural Information Processing Systems (NeurIPS) 30 (2017) (pp. 119–129). Curran Associates.

Buchman, E. (2016). *Tendermint: Byzantine fault tolerance in the age of blockchains* [Master's thesis, University of Guelph].

Castro, M., & Liskov, B. (1999). Practical Byzantine fault tolerance. In *Proceedings of the 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)* (pp. 173–186). USENIX Association.

Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. (2021). *Evaluating large language models trained on code* (arXiv:2107.03374). https://arxiv.org/abs/2107.03374

CISQ (Consortium for Information & Software Quality). (2022). The cost of poor software quality in the US: A 2022 report. https://www.it-cisq.org/the-cost-of-poor-quality-software-in-the-us-a-2022-report/

Clement, A., Wong, E., Alvisi, L., Dahlin, M., & Marchetti, M. (2009). Making Byzantine fault tolerant systems tolerate Byzantine faults. In *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation (NSDI)* (pp. 153–168). USENIX Association.

Dietterich, T. G. (2000). Ensemble methods in machine learning. In Multiple Classifier Systems (MCS), Lecture Notes in Computer Science (Vol. 1857, pp. 1-15). Springer. https://doi.org/10.1007/3-540-45014-9_1

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2024). Improving factuality and reasoning in language models through multi-agent debate. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Dwork, C., Lynch, N., & Stockmeyer, L. (1988). Consensus in the presence of partial synchrony. *Journal of the ACM, 35*(2), 288–323.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., & Zhou, M. (2020). CodeBERT: A pre-trained model for programming and natural languages. In Findings of the Association for Computational Linguistics: EMNLP 2020 (pp. 1536–1547). ACL. https://doi.org/10.18653/v1/2020.findings-emnlp.139

Fischer, M. J., Lynch, N. A., & Paterson, M. S. (1985). Impossibility of distributed consensus with one faulty process. *Journal of the ACM, 32*(2), 374–382. https://doi.org/10.1145/3149.214121

Gueta, G. G., Abraham, I., Grossman, S., et al. (2019). SBFT: A scalable and decentralized trust infrastructure. In *Proceedings of the 49th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)* (pp. 568–580). IEEE. https://doi.org/10.1109/DSN.2019.00063

Gupta, R., Pal, S., Kanade, A., & Shevade, S. (2017). DeepFix: Fixing common C language errors by deep learning. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence* (pp. 1345–1351). AAAI Press. https://doi.org/10.1609/aaai.v31i1.10742

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., et al. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)* (Oral). OpenReview.

Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)* (pp. 437–440). ACM. https://doi.org/10.1145/2610384.2628055

Kotla, R., Alvisi, L., Dahlin, M., Clement, A., & Wong, E. (2007). Zyzzyva: Speculative Byzantine fault tolerance. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)* (pp. 45–58). ACM. https://doi.org/10.1145/1294261.1294267

Lamport, L., Shostak, R., & Pease, M. (1982). The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems, 4*(3), 382–401. https://doi.org/10.1145/357172.357176

Le Goues, C., Nguyen, T., Forrest, S., & Weimer, W. (2012). GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering, 38*(1), 54–72. https://doi.org/10.1109/TSE.2011.104

Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D., & Ghanem, B. (2023). CAMEL: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS) 36*. Curran Associates.

Marginean, A., Bader, J., Chandra, S., Harman, M., Jia, Y., Mao, K., et al. (2019). SapFix: Automated end-to-end repair at scale. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)* (pp. 269–278). IEEE. https://doi.org/10.1109/ICSE-SEIP.2019.00039

Maturi, M. H., De La Cruz, E., Addula, S. R., Yadulla, A. R., Kathalikkattil Ravindran, R., Nadella, G. S., Gonaygunta, H., & Meduri, K. (2025). Enhancing smart contract security with explainable AI: A framework for re-entrancy vulnerability detection and explanation. In 2025 IEEE Systems and Information Engineering Design Symposium (SIEDS). IEEE. https://doi.org/10.1109/SIEDS65500.2025.11021147

Mechtaev, S., Yi, J., & Roychoudhury, A. (2016). Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)* (pp. 691–701). ACM. https://doi.org/10.1145/2884781.2884807

Miller, A., Xia, Y., Croman, K., Shi, E., & Song, D. (2016). The Honey Badger of BFT protocols. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)* (pp. 31–42). ACM. https://doi.org/10.1145/2976749.2978399

Nguyen, H. D. T., Qi, D., Roychoudhury, A., & Chandra, S. (2013). SemFix: Program repair via semantic analysis. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)* (pp. 772–781). IEEE. https://doi.org/10.1109/ICSE.2013.6606623

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., et al. (2024). ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. ACL. https://doi.org/10.18653/v1/2024.acl-long.810

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS) 36*.

Smith, E. K., Barr, E. T., Le Goues, C., & Brun, Y. (2015). Is the cure worse than the disease? Overfitting in automated program repair. In *Proceedings of the 10th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)* (pp. 532–543). ACM. https://doi.org/10.1145/2786805.2786825

Veronese, G. S., Correia, M., Bessani, A. N., Lung, L. C., & Verissimo, P. (2013). Efficient Byzantine fault-tolerance. *IEEE Transactions on Computers, 62*(1), 16–30. https://doi.org/10.1109/TC.2011.221

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2023). Self-consistency improves chain-of-thought reasoning in language models. In Proceedings of the 11th International Conference on Learning Representations (ICLR).

Widyasari, R., Sim, S. Q., Lok, C., Qi, H., Phan, J., Tay, Q., et al. (2020). BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)* (pp. 1556–1560). ACM. https://doi.org/10.1145/3368089.3417943

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., et al. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents* (Best Paper). OpenReview.

Xia, C. S., & Zhang, L. (2022). Less training, more repairing please: Revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)* (pp. 959–971). ACM.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*.

Yin, M., Malkhi, D., Reiter, M. K., Gueta, G. G., & Abraham, I. (2019). HotStuff: BFT consensus with linearity and responsiveness. In *Proceedings of the 38th ACM Symposium on Principles of Distributed Computing (PODC)* (pp. 347–356). ACM. https://doi.org/10.1145/3293611.3331591

Zhang, Y., Ruan, H., Fan, Z., & Roychoudhury, A. (2024). AutoCodeRover: Autonomous program improvement. In Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). ACM. https://doi.org/10.1145/3650212.3680384

Response to Reviewers — Manuscript SO-26-5308 (R1)

We thank the editor and both reviewers for a careful and constructive reading. The revision makes four substantive additions in direct response: (1) we implement and empirically evaluate a semantic-equivalence quorum, so the central claim is now demonstrated rather than deferred; (2) we add a cross-language evaluation on Java, including the real Defects4J harness with its actual JUnit test oracle; (3) we demonstrate the Byzantine fault tolerance of the consensus layer directly — independent replicas exchanging signed messages over an asynchronous, partitionable network preserve agreement under a malicious, equivocating primary where naive voting splits (Section 6.14) — together with run-to-run variance and latency analyses; and (4) we add a full reproduction protocol with exact models, seeds, prompts and a Docker definition. Point-by-point responses follow; section and table numbers refer to the revised manuscript. All changes are visible in the tracked-changes copy.

Editor's requirements

Ethics and informed-consent statements. Added to the title page: an Ethics statement and a separate Informed-consent statement. The study uses only publicly available open-source defects and author-constructed code; it involves no human or animal subjects, so approval and consent were not required, consistent with APA Section 8.05. The title page now also carries author contributions and ORCID fields.

Footnotes. The manuscript contains no footnotes; nothing needed to be moved into the body.

Anonymisation. The main document is anonymised: author names, affiliation, e-mail addresses, repository usernames and the named institution have been removed or replaced with neutral text. Author-identifying details remain only on the separate title page.

Language editing. The manuscript was edited for grammar and clarity (Grammarly plus a manual pass).

Reviewer 1

1. The semantic-equivalence quorum was claimed but deferred; experiments used exact match, so Gap 2 was unproven. We agree this was the core weakness and have resolved it. We implemented a behavioural semantic-equivalence quorum (Section 5.10) and evaluated it head-to-head against the exact-match rule across heterogeneous

proposers (Section 6.9, Table 9). Across 70 bugs the exact rule reached quorum on 2.5–10% of cases, while the semantic rule reached quorum on 70–77% and agreed on a passing fix in 60%. The claim is now demonstrated, and we have rewritten the abstract, introduction, Section 2.2, Section 2.4 and Section 8.1 so the text matches what the prototype does.

2 and 8. Validation was Python-only; include Defects4J (Java) or narrow the scope. We added a cross-language evaluation (Section 6.10) and ran the **real Defects4J benchmark** through its own checkout/compile/test harness in Docker, against the projects' **actual JUnit suites** rather than a proxy oracle. The healer produces a minimal search-and-replace edit (not a whole-file rewrite) and is shown the failing test; every candidate is verified against the project's full relevant test suite, so a repair counts only if the suite passes. On commons-math the pipeline produced **two genuine, full-suite-verified repairs (Math-3, unanimous 4-of-4; Math-5, 3-of-4 with one validator dissenting on a fix that nevertheless passed the real suite)**, with consensus reached on all 9 evaluated bugs. On commons-lang the four independent validators reached consensus on 8 of 9 evaluated bugs, but none passed the full Lang suite — a concrete, honest measure of how hard real-world Java repair is, and in every case the executable suite prevented an unverified fix from being reported as a repair (Table 10). The real oracle cleanly separates two questions the synthetic and snippet-level studies cannot: *consensus robustness*, which the Byzantine fault-injection matrix proves, and *repair capability*, which on hard real Java bugs is modest and which we report honestly. We are also explicit about reproducibility: the healer is non-deterministic, so the specific set of repaired bugs varies between runs even with a pinned seed; what is invariant is safety (no fix is reported as a repair unless the real suite passes). We therefore report repair counts as single-run, seeded figures and release the seeds.

3. Latency scaling with n and during fault recovery. Added in Section 6.12 and Table 11. Median latency rises with the agent count (53.3 s at n = 4, 71.3 s at n = 7, 80.1 s at n = 10) and the 99th percentile grows faster (59.6 s, 78.0 s, 168.6 s) as the round waits for the slowest of the added validators, while safety and success are unchanged. The PBFT protocol itself adds under 2 ms per round; the growth is validator inference, not consensus. Under the five Byzantine scenarios consensus still formed every round from the honest validators (Section 6.6), so fault behaviour did not inflate decision latency; continuous-fault recovery timing remains future work (Section 8.2).

4. Engage the explainable-AI smart-contract re-entrancy work. Added to Section 2.3: we connect the plausibility-correctness gap to the case for explainable, executable verification in adjacent code-security settings and cite the suggested work (Maturi et al., 2025), with full bibliographic details now in the reference list.

5. Diverse LLMs produce different code, which contradicts an exact-match quorum. This is exactly the effect we now measure. Section 6.9 shows mean pairwise text

agreement of only 0.18–0.20 across heterogeneous proposers, which is why the exact rule almost never forms a quorum and why the semantic rule is necessary. The decorrelation also appears in the safety pipeline itself: the four cross-provider validators agree in near-lockstep on easy bugs (~1.00) but fall to 0.925 on the real BugsInPy bugs, with three genuine split votes (Section 7.2). The mechanism is no longer ambiguous.

6. The abstract overstates "adapting PBFT". The abstract is rewritten. It now states the system precisely: a Claude analyzer and a GPT-4o healer propose a fix that is voted on by four independent, model-diverse validators under PBFT (n = 3f + 1 = 4, quorum = 3), with a sandbox as the final gate; it reports the genuine fault-tolerance results (f = 1 across all five behaviours, f = 2 tight bound) and the separately evaluated semantic-equivalence quorum without overclaiming.

7. Clarify how consensus was reached using exact match across different LLMs. Clarified in Section 3.2: in the safety pipeline a single healer proposal is put to the four independent validators, and a byte digest binds their votes to that one proposal (standard PBFT), so "exact match" there refers to *proposal binding*, not to agreement among different fixes. Agreement among multiple independent proposals is the separate semantic-quorum study (Sections 5.10, 6.9).

Reviewer 2

1. Abstract restructure. Rewritten to the suggested arc: problem, aim, method, findings, meaning, and next steps.

2. Introduction. Adds a concrete e-commerce failure hook, separates the practical from the scientific problem, refers to Table 1 rather than listing all five gaps in prose, and states the aim and the four research questions explicitly. We also note that RQ1/RQ2 are answered on a limited n-sweep.

3. Repeatability and reliability. Added a reproduction section (Section 5.9) with exact model strings — analyzer claude-sonnet-4-5-20250929 (temperature 0.3, no vote), healer/ proposer gpt-4o (0.7, no vote), and the four independent validators (Claude-Haiku and GPT-4o-mini in the cloud, llama3.1:8b and mistral:7b served locally by Ollama, all at 0.1) — with token budgets, timeouts and seed handling (we pin seeds for OpenAI and Ollama; Anthropic exposes none). The Docker sandbox image and limits are specified. A supplementary file gives the full command sequence, the verbatim agent prompts, and a BugsInPy case-to- project mapping. On single-run-per-bug: we add a variance study repeating a ten-bug subset three times (Section 6.11). The consensus pipeline was identical across all three runs (success 1.000, zero safety violations); only the single-agent baseline's raw success moved (1.00, 1.00, 0.90), so the outcomes the safety claim rests on do not vary run to run. Test adequacy: the synthetic and e-

commerce canonical fixes were each verified by hand against the bundled assertion suite.

4. Discussion depth. Section 7.1 now connects the result to ensemble learning (Dietterich, 2000), self-consistency (Wang et al., 2023) and debate (Du et al., 2024), and to overfitting in repair (Smith et al., 2015), while making clear that BFT-MAS adds what those lack: a genuine PBFT quorum over independent validators that bounds how many confidently wrong agents the system absorbs (tolerating f of 3f + 1). Section 7.2 expands the decorrelation result with three readings (difficulty ceiling, model convergence, prompt-induced correlation) and links it to the new quorum and split-vote data. A new Section 7.5 frames the economic and societal implications for managers and for public trust.

5. Conclusion and release. The conclusion now restates the scientific problem, the aim, and how the results resolve it, and lists the concrete advances. Prompts, raw CSVs, the BugsInPy mapping and the sandbox/Docker definition are released; an anonymised archive is available to reviewers and a public DOI will be added on acceptance.

6. Minor issues. We checked the manuscript: the "BugsInPy" and "llama3.1" spellings are correct throughout this version and Table 1 contains no stray markup (the artefacts the reviewer saw were in the submitted PDF rendering). All in-text citations were checked against the reference list, and Dietterich (2000) and Wang et al. (2023) were added. All figures are present in the source; we will verify the PDF export preserves them.

Additional changes (not specifically requested)

- **Byzantine fault tolerance, demonstrated directly (Section 6.14).** We were careful not

to overclaim here. The fault-injection matrix (Section 6.6) and the f = 2 study (Section 6.13) corrupt how a validator *votes*, which a majority quorum masks — useful, but not by itself the defining Byzantine property. To demonstrate that property we built an independent-replica realisation of the protocol: four replicas exchanging Ed25519-signed pre-prepare/prepare/commit messages over an asynchronous, partitionable network, and we pit it against a naive proposal-trust scheme. Under a **malicious primary that equivocates** (proposing fix X to some replicas and fix Y to others), the naive scheme splits — honest replicas commit different fixes (a safety violation) — while the protocol's 2f + 1 quorum certificates keep every honest replica in agreement (no split-brain). Forged messages are rejected (a replica cannot impersonate another, so one Byzantine node cannot manufacture a quorum), f = 1 crash is tolerated while f + 1 is not, and a partition blocks progress without ever causing a split, recovering on heal (Section 6.14, Table 13, Figure 7). The simulation is deterministic and uses no model calls. We are explicit that liveness under primary failure (view-change) and a multi-host deployment are

implemented or designed but not yet evaluated (Section 8), and that the quality of the LLM repairs the protocol agrees on is a separate, preliminary question.

- **Statistics.** For the binary safety outcome we use McNemar's exact test as the primary

test. All discordant pairs fall in the 20-bug BugsInPy subset, where the safety-violation rate fell from 75% to 50% (discordant pairs b = 6, c = 1; McNemar exact p = 0.125) and the repair rate rose from 25% to 50% (discordant 6 to 1; p = 0.125); both are directional and do not reach significance at this sample size, which we state plainly (across all 90 paired bugs the violation rate is 16.7% for the baseline against 11.1% for consensus). Accordingly we rest the paper's central reliability claim on the deterministic Byzantine-tolerance results above, which are provable rather than sampled, and report the paired repair and safety comparisons as directional. We also note the clustering of the real-world pairs (all discordant pairs in BugsInPy, concentrated in a few projects), so the effective sample is smaller than the raw count (Section 7.3).

- **Repair capability, reported honestly.** On the 20-bug BugsInPy subset the four

independent validators admit more genuine fixes than the single agent (10 of 20 against 5 of 20), not fewer; because the BugsInPy oracle is a weak run-as-script check (Section 5.2), consensus also approves fixes the sandbox then rejects, but every such fix is caught before it could be reported as a repair. We make this construct-validity limitation explicit and tie the headline numbers to it.

- **Cross-language evaluation with a real oracle.** As noted under Reviewer 1, the Defects4J

runs use the projects' real JUnit suites; the two verified commons-math repairs (Math-3, Math-5) and the modest commons-lang outcome are reported with no unverified fix counted as a repair.

- **Reproducibility configuration, threat model and scope (Section 3.2).** A new

"Threat model and scope" paragraph states that the four validators run as asynchronous tasks in one trusted process, that the adversary modelled is a Byzantine validator injected by a wrapper, and that the per-message signatures protect integrity by construction. Within this model the consensus is genuine 3f + 1 Byzantine fault tolerance evaluated in a single-process simulation; what remains future work is the *distributed* setting — real inter-host messaging, network partitions and view-change under primary failure (Section 8.5).

- **Internal-consistency reconciliation.** Table 2's BFT-MAS row now reads Byzantine

Consensus "Yes (simulated)", Cross-Language "Partial (Java sample)", Production Ready "No (research prototype)", and Knowledge Graph / Learning Capability "Implemented, not evaluated"; the metrics contribution lists six measured metrics plus pairwise agreement (recovery time deferred); §5.6 metric 7 is relabelled "ground-truth accuracy" to match

Table 4; the §7.5 economic figures use a single "per 100 repairs" base; and the duplicated Acknowledgements / Declarations were removed from the main document (they remain on the Title Page).

- **Decorrelation.** We report mean pairwise validator agreement as a first-class result

(Section 7.2): ~1.00 on easy synthetic bugs, 0.925 on the real BugsInPy bugs (with three genuine split votes), and 0.54 among heterogeneous proposers in the semantic-quorum study, directly addressing the agent-diversity gap.

- **The BugsInPy oracle is made explicit (§5.2).** The projects' own test suites are not

run; the diff-extracted candidate is executed in the sandbox (a clean exit counts as a pass) and compared to the canonical fixed hunk. §7.3 flags this snippet-level oracle as the main construct-validity threat and notes the headline repair numbers rest on it.

- **References.** Added CISQ (2022), CodeBERT (Feng et al., 2020), AutoCodeRover (Zhang et

al., 2024), Dietterich (2000), Wang et al. (2023) and Maturi et al. (2025); the Table 2 "GitHub Copilot" row now cites Codex (Chen et al., 2021); the Blanchard et al. entry is corrected to 2017; and the reference list is now fully alphabetical with DOIs backfilled.

- **Figures.** Figure 1 is redrawn to the genuine pipeline (Claude analyzer and GPT-4o

proposer, neither voting; four independent validators voting under a 3-of-4 quorum); Figures 2, 4 and 5 are regenerated with the genuine-3f + 1 framing and honest statistics; Figure 6 (the f = 2 vote cliff) is new.

Supplementary material: reproduction protocol and agent prompts

Manuscript SO-26-5308. This file accompanies the revised submission and responds to Reviewer 2, points on repeatability (3.1, 3.2) and release (5). The consensus layer is genuine 3f + 1 Byzantine fault tolerance: a Claude analyzer and a GPT-4o healer propose a fix that is voted on by four independent, model-diverse validators (n = 3f + 1 = 4, quorum = 2f + 1 = 3, f = 1); the analyzer and healer do not vote.

1. Exact model and decoding configuration

Agent	Role	Provider	Model string	Temp	Max tokens	Timeout	Seed
Analyzer	feeds diagnosis (no vote)	Anthropic	claude-sonnet-4-5-20250929	0.3	2048	120 s	n/a (no seed param)
Healer	proposer (no vote)	OpenAI	gpt-4o	0.7	2048	120 s	pinned (OpenAI `seed`)
Validator 1	votes	Anthropic	claude-haiku-4-5-20251001	0.1	512	120 s	n/a (no seed param)
Validator 2	votes	OpenAI	gpt-4o-mini	0.1	512	120 s	pinned (OpenAI `seed`)
Validator 3	votes	Ollama	llama3.1:8b	0.1	512	300 s	pinned (`options.seed`)
Validator 4	votes	Ollama	mistral:7b	0.1	512	300 s	pinned (`options.seed`)

The four validators are model-diverse and cross-provider (two cloud, two local). Independence rule: no validator is the GPT-4o proposer, and the Claude validator is Haiku, distinct from the Sonnet analyzer. `top_p`, `frequency_penalty` and `presence_penalty` are left at provider defaults. The f = 2 tight-bound study (Section 6.13) uses seven homogeneous llama3.1:8b validators (n = 3f + 1 = 7, quorum = 2f + 1 = 5). Sandbox: Docker, network disabled, 512 MB memory, 1 CPU; python:3.11-slim (Python) and eclipse-temurin:17-jdk (Java); isolated-subprocess fallback when Docker is absent. Hardware: HP EliteBook 865 G11, AMD Ryzen 7 8840HS, 64 GB RAM, Windows 11, no GPU (Ollama validators run on the CPU, which is why their per-call timeout is larger).

2. Reproduction commands

```
# environment
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
ollama pull llama3.1:8b
ollama pull mistral:7b
# set ANTHROPIC_API_KEY and OPENAI_API_KEY in .env

# main consensus runs (genuine 3f+1 heterogeneous panel) vs single-agent baseline
python -m benchmarks --dataset synthetic --pipeline real --sandbox-backend subprocess --
mode consensus --hetero-validators --limit 40 --max-attempts 3 --case-timeout-s 600
python -m benchmarks --dataset synthetic --pipeline real --sandbox-backend subprocess --
mode single-agent --limit 40 --max-attempts 3
python -m benchmarks --dataset bugsinpy --pipeline real --sandbox-backend subprocess --mode
consensus --hetero-validators --bugsinpy-root data/bug_datasets/bugsinpy --bugsinpy-
projects PySnooper ansible --limit 20 --max-attempts 3 --case-timeout-s 600
python -m benchmarks --dataset bugsinpy --pipeline real --sandbox-backend subprocess --mode
single-agent --bugsinpy-root data/bug_datasets/bugsinpy --bugsinpy-projects PySnooper
ansible --limit 20 --max-attempts 3
python -m benchmarks --dataset ecommerce --pipeline real --sandbox-backend subprocess --
mode consensus --hetero-validators --limit 30 --max-attempts 3

# Byzantine fault injection (f=1, one of four independent validators corrupted)
for fault in always_reject always_approve random timeout garbage; do
  python -m benchmarks --dataset ecommerce --pipeline real --sandbox-backend subprocess --
mode consensus --hetero-validators \
    --inject-fault $fault --inject-fault-count 1 --limit 10 --max-attempts 1
done

# f=2 tight bound (n=7, quorum=5; homogeneous llama3.1 replicas): tolerate 2, break at
f+1=3
python -m benchmarks --dataset ecommerce --pipeline real --sandbox-backend subprocess --
mode consensus --f 2 --n-validators 7 --inject-fault always_reject --inject-fault-count 2 -
-limit 5
python -m benchmarks --dataset ecommerce --pipeline real --sandbox-backend subprocess --
mode consensus --f 2 --n-validators 7 --inject-fault always_reject --inject-fault-count 3 -
-limit 5

# latency n-sweep (independent voters, f=1)
for n in 4 7 10; do
  python -m benchmarks --dataset ecommerce --pipeline real --sandbox-backend subprocess --
mode consensus --f 1 --n-validators $n --limit 5 --max-attempts 1
done

# semantic vs exact quorum study (four heterogeneous Ollama proposers, quorum=3)
python -m benchmarks.semantic_quorum --dataset synthetic --limit 40 --seed 7
python -m benchmarks.semantic_quorum --dataset ecommerce --limit 30 --seed 7

# real Defects4J (Java), after building the defects4j Docker image; real JUnit oracle
python -m benchmarks.defects4j_runner --project Lang --hetero-validators --mode consensus -
-bugs 1,2,3,4,5,6,7,8,9,10
python -m benchmarks.defects4j_runner --project Math --hetero-validators --mode consensus -
-bugs 1,2,3,4,5,6,7,8,9,10
```

LLM agents are non-deterministic (Anthropic exposes no seed; cloud providers do not guarantee bit-exact decoding), so an independent rerun will differ in detail. We pin the OpenAI and Ollama seeds and release them; what is invariant across runs is safety — no fix is reported as a repair unless it passes the executable check.

3. Expected headline outcomes (for sanity-checking a rerun)

- **Byzantine tolerance (f = 1):** consensus formation holds across all five fault behaviours with one of four validators corrupted; liveness holds under always-reject and timeout, safety holds under always-approve and random.
- **Tight bound (f = 2, n = 7, quorum = 5):** consensus forms with two Byzantine rejecters and provably fails with three (prepare votes 7 -> 5 -> 4).
- **Repair (BugsInPy, 20 bugs):** consensus repairs 10/20, single-agent 5/20; recall 1.0 against the sandbox oracle, with three genuine split votes (ansible-1, -12, -14; Claude-Haiku dissenting).
- **Defects4J (real JUnit oracle):** two full-suite-verified commons-math repairs (Math-3, Math-5); commons-lang consensus reached on 8/9 evaluated bugs with no full-suite pass; no unverified fix counted as a repair.
- **Quorum rule:** exact-match quorum forms on 2.5-10% of bugs, semantic-equivalence quorum on 70-77% (passing fix in 60%); mean pairwise text agreement 0.18-0.20, behavioural 0.54.
- **Decorrelation:** mean pairwise validator agreement ~1.00 on easy synthetic bugs, 0.925 on real BugsInPy bugs.

4. BugsInPy case mapping (n=20)

Case ID	Project
bip-PySnooper-1	PySnooper
bip-PySnooper-2	PySnooper
bip-PySnooper-3	PySnooper
bip-ansible-1	ansible
bip-ansible-2	ansible
bip-ansible-3	ansible
bip-ansible-4	ansible
bip-ansible-5	ansible
bip-ansible-6	ansible
bip-ansible-7	ansible
bip-ansible-8	ansible
bip-ansible-9	ansible
bip-ansible-10	ansible
bip-ansible-11	ansible
bip-ansible-12	ansible
bip-ansible-13	ansible
bip-ansible-14	ansible
bip-ansible-15	ansible
bip-ansible-16	ansible

bip-ansible-17	ansible

5. Agent prompts (verbatim templates)

5.1 Analyzer (Claude Sonnet, no vote)

```
You are an expert software engineer analyzing a runtime error.
Analyze the following bug and provide a structured analysis.

## Error Information
**Error Message:** <error>

**File:** main.py
**Line:** 3
**Language:** python

## Stack Trace
```

<trace>

```
## Code Context
```

<buggy code>

```
## Your Task
Provide a JSON response with the following structure:
{
    "bug_type": "<one of: null_pointer, type_error, index_out_of_bounds, division_by_zero,
api_error, timeout, memory_error, syntax_error, logic_error, concurrency_error, unknown>",
    "severity": "<one of: critical, high, medium, low>",
    "root_cause": "<brief explanation of why this error occurred>",
    "affected_code": "<the specific code causing the issue>",
    "suggested_approach": "<how to fix this bug>",
    "related_patterns": ["<list of similar bug patterns>"]
}

Respond ONLY with the JSON object, no additional text.
```

5.2 Healer / proposer (GPT-4o, no vote)

```
You are an expert software engineer fixing a bug in production code.
Generate multiple fix candidates for the following bug.

## Bug Analysis
**Type:** off-by-one
**Severity:** high
**Root Cause:** loop bound
**Suggested Approach:** fix the bound

## Original Code
**File:** main.py
**Line:** 3
```

```
**Language:** python
```

<buggy code>

```
## Your Task
Generate 2-3 different fix candidates. For each fix:
1. Provide the complete fixed code
2. Explain what was changed and why
3. Estimate confidence (0.0-1.0)
4. List potential side effects

Respond with a JSON object:
{
    "candidates": [
        {
            "fixed_code": "<complete fixed code>",
            "explanation": "<what was changed and why>",
            "confidence": <0.0-1.0>,
            "changes_description": "<brief description of changes>",
            "potential_side_effects": ["<list of potential issues>"]
        }
    ],
    "recommended_index": <index of best candidate>,
    "generation_strategy": "<brief description of approach>"
}

Important:
- Preserve the original code structure
- Only fix the actual bug, don't refactor
- Ensure the fix is safe for production
- Consider edge cases

Respond ONLY with the JSON object.
```

5.3 Validator (used by all four independent validators)

```
You are reviewing a candidate bug fix as part of a Byzantine consensus stage. The fix has
already been generated by a Healer agent and will be executed in an isolated sandbox after
this review. Your job is structural safety review, NOT exhaustive correctness proof.

## Original Code (buggy)
```

<buggy code>

```
## Proposed Fix
```

<candidate fix>

```
## Fix Explanation
...

## Voting Rules
- Vote IS_VALID = true if the fix plausibly addresses the bug AND introduces no obvious
```

```
safety problem.
- Vote IS_VALID = false ONLY if you can name a SPECIFIC safety problem (concrete line,
condition, or scenario where the fix would crash, corrupt data, or violate an invariant).
- Style preferences, naming choices, missing comments, suboptimal performance, or "could be
cleaner" critiques are NOT grounds for rejection. Record them under recommendations.
- If the fix is plausibly correct but you are uncertain, vote IS_VALID = true with moderate
confidence (0.5-0.7). The sandbox stage will catch true failures.

Respond with JSON only:
{
    "is_valid": <true|false>,
    "confidence": <0.0-1.0>,
    "issues": ["<specific safety problem 1>", "<specific safety problem 2>"],
    "recommendations": ["<style/improvement suggestion 1>"]
}

The "issues" list should be EMPTY if the fix is plausibly safe. Only populate it with
concrete, specific problems.
```